Spring Handbook — Interview Edition

How to use this edition

The night-before version of the Spring Handbook: every chapter's key ideas, comparison tables, likely questions and traps, with the explanations and code walkthroughs stripped out. Everything here comes from the full chapter notes (`../chapters/NN-*/README.md`); go there when a line needs its "why". Versions are those of the book: **Spring Boot 4.1.1** → Framework 7.0.9, Security 7.1.1, Spring Data 2026.0, Hibernate 7.4, Jackson 3.1, JUnit 6, Java 25 (17 minimum); separately versioned: Spring Cloud 2025.1.x "Oakwood", Spring AI 2.0.1.

- **Must-know** — the terms and rules you must be able to say in one breath.
- **Key facts & numbers** — comparison tables (scopes, lifecycle, bean resolution, conditions, proxies, propagation, fetch strategies, slices, Reactor operators, Kafka error handling, Actuator, Spring Cloud, Boot 3 → 4).
- **Code to remember** — only the few snippets worth writing on a whiteboard.
- **In the interview** — the most likely questions (mostly from the book's 168-question Chapter 18) with 1–3 sentence answers. Say the answer out loud before reading it.
- **Gotchas** — the traps: "wrong idea → right idea".
- **TL;DR** — the chapter in one or two lines.

Suggested passes. 30 minutes: Top 25 questions + every TL;DR + Last-minute checklist. 2 hours: all Key facts tables + In the interview. Most "why doesn't this work?" questions live in Chapters 2, 3, 4, 7 and 10 — reread those first.

How to answer a Spring question (three layers). (1) One sentence of what it is — "`@Transactional` starts and ends a transaction around a method." (2) How it works underneath — "a proxy's interceptor asks the transaction manager to begin, calls the method, then commits or rolls back." (3) The consequence that bites — "so self-invocation skips it, and a checked exception commits by default." Layer 3 is what makes you sound experienced.

Top 25 questions across the book

#	Question	One-line answer	Ch
1	Spring vs Spring Boot?	Framework = IoC container + modules; Boot adds starters, a version BOM, auto-configuration, embedded server, externalized config, Actuator.	1
2	What does `@SpringBootApplication` do?	`@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan` from the main class's package down.	1
3	Two beans of one type — which is injected?	`@Qualifier` → `@Primary` → the one non-`@Fallback` → parameter name → else `NoUniqueBeanDefinitionException`.	2
4	Prototype injected into a singleton?	Created once and kept; use `ObjectProvider.getObject()`, `@Lookup` or a scoped proxy.	2
5	Bean lifecycle order?	Construct → inject → Aware → BPP-before → `@PostConstruct` → `afterPropertiesSet` → init-method → BPP-after (proxies) → … → `@PreDestroy` → `destroy` → destroy-method.	2
6	Circular dependency?	Constructor cycles always fail; Boot rejects field cycles since 2.6; extract a third bean or use an event (`@Lazy` as a patch).	2
7	How does auto-configuration work?	`.imports` list → early class filter → sorted → processed after your config → each guarded by `@ConditionalOn…`, usually `@ConditionalOnMissingBean`.	3
8	Why was a bean (not) auto-configured?	`--debug` condition evaluation report or `/actuator/conditions`.	3
9	What is self-invocation?	`this.method()` bypasses the proxy, so `@Transactional`, `@Cacheable`, `@Async`, `@Retryable` are silently skipped.	4
10	JDK proxy vs CGLIB?	Interface-based vs subclass; Boot uses CGLIB by default; CGLIB can't advise `final`/`private` methods.	4
11	Walk through an MVC request.	Filters → `DispatcherServlet` → `HandlerMapping` → interceptors → `HandlerAdapter` (args, body, validation) → controller → converter → exception resolvers.	5
12	Consistent error responses?	One `@RestControllerAdvice` extending `ResponseEntityExceptionHandler` returning RFC 9457 `ProblemDetail`.	5
13	Which HTTP client in Boot 4?	`RestClient` (or an `@HttpExchange` interface on it); `WebClient` only for reactive; `RestTemplate` is legacy.	6
14	Checked exception in `@Transactional` — commit or rollback?	Commit; only `RuntimeException`/`Error` roll back unless `rollbackFor` or `rollbackOn = ALL_EXCEPTIONS`.	7
15	`UnexpectedRollbackException`?	An inner `REQUIRED` call failed and marked the shared transaction rollback-only; the outer caught it and tried to commit.	7
16	N+1 and its fixes?	1 query + 1 per parent's lazy association; `JOIN FETCH`, `@EntityGraph`, batch size, DTO projection.	8
17	`LazyInitializationException`?	Touching a lazy proxy after the persistence context closed; fetch in the query inside the transaction, return DTOs.	8
18	Why doesn't `@Cacheable` work?	No `@EnableCaching`, self-invocation, non-public method, keys without `equals`/`hashCode`.	9
19	Explain the Security filter chain.	`DelegatingFilterProxy` → `FilterChainProxy` → first matching `SecurityFilterChain` → context, CORS/CSRF, authentication, `ExceptionTranslationFilter`, `AuthorizationFilter`.	10
20	401 vs 403?	401 = not authenticated ("who are you?"); 403 = authenticated but not allowed (also a missing CSRF token).	10
21	How does a resource server validate a JWT?	Fetch JWKs from the issuer, pick key by `kid`, verify signature, check `exp`/`nbf`/`iss`(/`aud`), map `scope` → `SCOPE_*`.	11
22	`@MockBean` doesn't compile in Boot 4?	Removed in 4.0; use `@MockitoBean` / `@MockitoSpyBean` (Framework 6.2).	12
23	WebFlux or MVC + virtual threads?	MVC + virtual threads for CRUD with JDBC/JPA; WebFlux for streaming and end-to-end non-blocking. Both ~8,000 req/s in the book's test.	13

#	Question	One-line answer	Ch
24	A Kafka message keeps failing?	`DefaultErrorHandler` with back-off + `DeadLetterPublishingRecoverer` → `<topic>-dlt`; non-retryable exceptions straight to DLT; `@RetryableTopic` for non-blocking retries.	14
25	Boot 3 → 4 migration?	3.5.x first with no deprecations; rename starters + test starters; Jackson 3; `@MockitoBean`; `@AutoConfigure…` for test clients; Spring Cloud 2025.1.x, Spring AI 2.0.	17

1. The Spring Ecosystem and Your First App

What Spring and Boot are, versions, project layout, starters, `@SpringBootApplication`, running the jar, DevTools, a first endpoint and tests.

Must-know

- **Spring Framework** — an IoC container that creates and wires your objects (**beans**) + modules for web, data, transactions, messaging.
- **Spring Boot** — Spring + **starters** + a managed version **BOM** + **auto-configuration** + **embedded server** + externalized config + Actuator. Boot removes boilerplate; it does not replace Spring.
- **Starter** — a dependency with no code that brings a curated, version-aligned library set for one capability plus the Boot module holding its auto-configuration.
- `@SpringBootApplication` — `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan` (main class's package **and below**).
- **Fat jar** — manifest `Main-Class` = `org.springframework.boot.loader.launch.JarLauncher`, your class = `Start-Class`; libraries in `BOOT-INF/lib`.
- `SpringApplication.run` — detects app type → prepares the `Environment` → creates the `ApplicationContext` → registers scanned beans + matching auto-configurations → refresh (singletons, embedded Tomcat) → runs `CommandLineRunner` / `ApplicationRunner`.
- **DevTools** — fast restart via two class loaders on recompile; disabled for packaged jars and tests; LiveReload off in Boot 4.
- **Jakarta** — Java EE moved to Eclipse, so packages are `jakarta.*` (not `javax.*`) since Spring 6.

Key facts & numbers

Item	Value in this book
Generation	Framework 7 and Boot 4 (both Nov 2025); Boot **4.1.1** → Framework **7.0.9**
Java	17 minimum, **25** used/recommended
Platform	Jakarta EE 11: Servlet 6.1, JPA 3.2, Bean Validation 3.1; Tomcat 11, Jetty 12.1; **Undertow removed**
Libraries	Jackson 3 (`tools.jackson`), JUnit 6, Hibernate 7.4, Security 7.1.1
Boot 4 starter renames	`-web` → `spring-boot-starter-webmvc`; `-aop` → `-aspectj`; a `-test` starter per technology (`spring-boot-starter-webmvc-test`)
Both MVC and WebFlux starters present	Boot picks **Spring MVC** (servlet)
Graceful shutdown	on by default

In the interview

Q: Walk me through what happens when you run a Spring Boot jar. A: `JarLauncher` builds a class loader over `BOOT-INF/lib` and calls `Start-Class`; `SpringApplication.run` prepares the `Environment`, creates the context, registers beans and matching auto-configurations, refreshes (singletons + embedded Tomcat), then runs the runners.

Q: Why an embedded server? Can you still build a WAR? A: One self-contained artifact everywhere, easy to containerise, no server-version drift. WAR: packaging `war`, extend `SpringBootServletInitializer`, mark the server starter `provided`.

Q: Can you override what Boot configures? A: Yes — auto-configured beans are guarded by `@ConditionalOnMissingBean`, so your own bean wins; many settings are just properties.

Q: How would you scan a package outside the main package? A: Move code under the root package (best), or `scanBasePackages = "..."` / `@Import`.

Q: What changed in Boot 4 / Framework 7? A: Java 17 baseline, Jakarta EE 11, modular starters, Jackson 3, JUnit 6, JSpecify, API versioning, HTTP service clients, `RestTestClient`, core `@Retryable`/`@ConcurrencyLimit`, Undertow removed (Ch 17).

Gotchas

- Adding `<version>` to a Boot-managed dependency → overrides the tested version; let the parent's BOM decide.
- Main class in a sub-package → sibling controllers aren't scanned → 404s. Never use the default package.
- `@Controller` for a JSON API → the return value is treated as a **view name**; use `@RestController`.
- Adding Spring Data JPA "just in case" → "Failed to configure a DataSource"; every starter can switch on auto-configuration.
- `spring-boot-starter-test` alone in Boot 4 → `@WebMvcTest` missing; add `spring-boot-starter-webmvc-test`.
- Logic in `main` after `run(...)` "before the server starts" → wrong; use a runner or event listener.

TL;DR

Boot = Spring + starters + BOM + auto-configuration + embedded server. Boot 4.1.1 → Framework 7.0.9, Jakarta EE 11, Jackson 3, JUnit 6, Java 17+ (25 here); modular starters (`-webmvc`, per-tech `-test`).

2. IoC and Dependency Injection

The container, defining beans, injection, how a bean is picked, scopes, lifecycle, post-processors, cycles, lazy init, conditions, events, JSpecify, `BeanRegistrar`.

Must-know

- **IoC** — the framework controls object creation and calls your code; **DI** — an object declares what it needs (constructor parameters) and the container supplies it.
- **Bean** — an object whose creation, wiring and lifecycle the container manages, described by a `BeanDefinition`.
- **`BeanFactory` vs `ApplicationContext`** — minimal lazy container vs full one: **eager singletons** (fail fast), auto-registered post-processors, events, `Environment`/profiles, i18n, resources.
- **`@Component` vs `@Bean`** — annotate your own class (found by scanning) vs factory method for third-party classes or creation logic. `@Repository` also enables exception translation.
- **Full vs lite `@Configuration`** — full (`proxyBeanMethods = true`) is a CGLIB subclass so inter-`@Bean` calls return the singleton; lite (`false`, or `@Bean` in a `@Component`) creates a second object. Pass dependencies as `@Bean` method parameters.
- **Constructor injection** — `final` fields, complete objects, visible dependencies, testable with `new`, cycles fail loudly; no `@Autowired` needed on a single constructor.
- **Singleton ≠ thread-safe** — one instance shared by all request threads; keep it stateless.
- **Scoped proxy** — `@RequestScope` injects a proxy that looks up the current request's instance per call; outside a request → `ScopeNotActiveException`.
- **`BeanFactoryPostProcessor`** edits **definitions** before any bean exists (`${…}` resolution); **`BeanPostProcessor`** sees each **instance** and can wrap it (`@Autowired`, AOP proxies). Declare both `static`.
- **Events** — `publishEvent` + `@EventListener`: **synchronous**, same thread, same transaction; `@Async` or `@TransactionalEventListener` to change that.
- **JSpecify** (Framework 7) — `@NullMarked` packages + `@Nullable` exceptions replace `org.springframework.lang`; compile-time tooling (NullAway, IntelliJ, Kotlin), no runtime checks.
- **`BeanRegistrar`** (Framework 7) — programmatic registration (loops, conditions), imported with `@Import`, AOT-friendly.

Key facts & numbers

Bean resolution order (proved):

Step	Rule
0	match by **type**
1	`@Qualifier("name")` on the injection point
2	`@Primary` bean
3	the only non-`@Fallback` bean (Framework 6.2)
4	bean name equals parameter name (needs `-parameters`, set by the Boot parent)
5	else `NoUniqueBeanDefinitionException` (none at all → `NoSuchBeanDefinitionException`)
All of them	`List<T>` (ordered by `@Order`, lowest first) or `Map<String,T>` (by bean name)
Optional	`ObjectProvider<T>`, `Optional<T>`, `@Nullable`, `@Autowired(required = false)`

Scopes:

Scope	Instances	Destroy callbacks?	Declare
`singleton` (default)	one per container, shared — stateless!	yes	nothing
`prototype`	new per `getBean`/injection	**no**	`@Scope("prototype")`
`request` / `session` / `application`	per HTTP request / session / `ServletContext`	yes	`@RequestScope` / `@SessionScope` / `@ApplicationScope` (proxies)
`websocket`	per WebSocket session	yes	`@Scope("websocket")`
refresh (Spring Cloud)	re-created after config refresh	yes	`@RefreshScope`

Lifecycle order (proved: init `[constructor, @PostConstruct, afterPropertiesSet, initMethod]`, destroy `[@PreDestroy, destroy(), destroyMethod]`, none for prototypes):

#	Step
1–2	constructor (constructor injection), then setter/field injection
3	`*Aware` callbacks (`BeanNameAware`, `ApplicationContextAware`)
4	`BeanPostProcessor.postProcessBeforeInitialization`
5	`@PostConstruct` → `afterPropertiesSet()` → `initMethod`
6	`postProcessAfterInitialization` — **AOP proxies created here**
7	`SmartLifecycle.start` → `ContextRefreshedEvent` → runners → `ApplicationReadyEvent`
8	shutdown: `ContextClosedEvent`, `SmartLifecycle.stop`
9	`@PreDestroy` → `destroy()` → `destroyMethod`

Hook	Use for
`@PostConstruct`/`@PreDestroy`	default: validate, warm up, release
`@Bean(initMethod, destroyMethod)`	third-party classes (public `close()`/`shutdown()` inferred)
`SmartLifecycle` (`getPhase()`)	things that run: pollers, listener containers
`@EventListener(ApplicationReadyEvent.class)`	"after fully started" work

Code to remember

```
@Service
class OrderService {                        // constructor injection, no @Autowired needed
    private final PaymentGateway gateway;
    private final ObjectProvider<Receipt> receipts;   // fresh prototype per call
    OrderService(@Qualifier("stripe") PaymentGateway gateway, ObjectProvider<Receipt> receipts) {
        this.gateway = gateway; this.receipts = receipts;
    }
    void pay() { Receipt r = receipts.getObject(); /* ... */ }
}
```

In the interview

Q: `@Primary` vs `@Qualifier`? A: `@Primary` is the provider's default; `@Qualifier` is the consumer's explicit choice at one injection point — the qualifier wins.

Q: You inject a prototype into a singleton. What happens? A: It is created once with the singleton and kept forever; use `ObjectProvider.getObject()`, `@Lookup` or a scoped proxy for a fresh one.

Q: `BeanPostProcessor` vs `BeanFactoryPostProcessor`? A: BFPP changes bean definitions before any bean is created; BPP modifies or wraps every instance (injection, AOP proxies).

Q: How does Spring handle circular dependencies? A: Constructor cycles → `BeanCurrentlyInCreationException`; field/setter cycles are rejected by Boot since 2.6 (`spring.main.allow-circular-references=false`). Fix the design (third bean, event); `@Lazy` on one parameter as a stop-gap.

Q: What does lazy initialization cost? A: Faster startup, but configuration errors appear at the first request and the first request is slower — risky for services.

Q: `@Profile` vs `@Conditional`? A: `@Profile` is a ready-made condition on active profiles; `@Conditional` takes any `Condition` — Boot's `@ConditionalOn…` are built on it.

Q: Are application events asynchronous? A: No — synchronous on the publisher's thread and inside its transaction; a listener exception reaches the publisher.

Gotchas

- `@Service` on an interface → nothing registered; annotate the implementation.
- Mutable field in a singleton service → race conditions under load.
- Request-scoped bean injected into a singleton without a proxy → "Scope 'request' is not active".
- `@Transactional` call from `@PostConstruct` → no transaction (proxy is created in step 6, and it's `this` anyway).
- Expecting `@PreDestroy` on prototypes, or after `kill -9` → never runs.
- `@Order` does not choose the single injected bean — it only orders collections.
- Two beans with the same name → `BeanDefinitionOverrideException` (overriding disabled in Boot).
- Heavy work in constructors/`@PostConstruct` → slow, fragile startup; use `ApplicationReadyEvent`.

TL;DR

Constructor injection; type → `@Qualifier` → `@Primary` → `@Fallback` → parameter name; singletons are shared and must be stateless; proxies are made in BPP-after-init; constructor cycles never work.

3. Configuration and Auto-configuration

Property sources and precedence, relaxed binding, `@Value` vs `@ConfigurationProperties`, profiles, config import, how auto-configuration decides, custom starters, Boot 4 modularization.

Must-know

- **`Environment`** — an ordered list of property sources; the first source that has a key wins.
- **Relaxed binding** — env var: `.` → `_`, drop `-`, upper-case: `SHOP_CHECKOUTTIMEOUT` → `shop.checkout-timeout`; list index `_0_`. `@Value` placeholders must use kebab-case to get relaxed matching.
- **`@Value`** — one value, supports SpEL. **`@ConfigurationProperties(prefix)`** — a typed group (a **`record`** is ideal): relaxed binding, nesting, `Duration`/`DataSize`, `@Validated`, IDE metadata. Register with `@ConfigurationPropertiesScan` or `@EnableConfigurationProperties` (not `@Component` for a record).
- **Fail fast on bad config** — `@Validated` + Jakarta constraints (`@Valid` on nested groups); startup stops and names each property and its origin.
- **Profiles** — named sets of beans and config (`application-prod.yaml`, `@Profile("prod")`), `spring.profiles.active`; **groups**: `spring.profiles.group.prod=postgres,kafka,json-logs`.

- `spring.config.import` — other files, `optional:configtree:/etc/secrets/` (Kubernetes secrets: file name = key), Config Server/Vault; `optional:` = don't fail if missing.
- **Auto-configuration** — `@AutoConfiguration` classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`; filtered early by class presence, sorted (`before`/`after`), processed **after** your configuration, guarded by conditions.
- **Override** — define your own bean (`@ConditionalOnMissingBean` backs off), set properties, or declare a customizer (`JsonMapperBuilderCustomizer`, `RestClientCustomizer`); switch one off with `exclude =` or `spring.autoconfigure.exclude`.
- **Custom starter** — `acme-spring-boot` (auto-configuration + properties + `@ConditionalOnMissingBean` on every bean, listed in `.imports`) + code-free `acme-spring-boot-starter` pom; test with `ApplicationContextRunner`.
- **Boot 4 modularization** — the single `spring-boot-autoconfigure` jar is split into per-technology modules (`spring-boot-webmvc`, `spring-boot-jdbc`, `spring-boot-kafka`…) each with a starter and a `-test` starter; `spring-boot-starter-classic` / `-test-classic` = migration aid.

Key facts & numbers

Precedence (high → low): test properties (`@TestPropertySource` > `@DynamicPropertySource` > `@SpringBootTest(properties)`) > **command line** > `SPRING_APPLICATION_JSON` > Java system properties > **OS env vars** > `random.*` > config data files > `@PropertySource` > defaults. Inside config data (high → low): profile file outside jar > plain file outside jar > profile file inside > plain file inside. Rule: *specific beats general; outside beats inside.*

Auto-configuration conditions:

Condition	Matches when	Typical use
`@ConditionalOnClass` / `OnMissingClass`	class is / isn't on the classpath	configure Jackson only if present
`@ConditionalOnBean` / `OnMissingBean`	bean exists / doesn't	**back off** when the user defined one
`@ConditionalOnSingleCandidate`	exactly one (or one primary) bean	`JdbcTemplate` if one `DataSource`
`@ConditionalOnProperty`	property has a value (`havingValue`, `matchIfMissing`)	feature switches
`@ConditionalOnBooleanProperty` (Boot 3.5+)	boolean property is `true`	`spring.jmx.enabled`
`@ConditionalOnResource`	resource exists	`META-INF/build-info.properties`
`@ConditionalOnWebApplication` / `OnNotWebApplication`	servlet/reactive web app or not	web-only beans
`@ConditionalOnThreading`	platform vs virtual threads	executors
`@ConditionalOnCloudPlatform`, `OnJava`, `OnWarDeployment`	platform, JVM range, WAR	platform tweaks
`@ConditionalOnExpression`	SpEL is true	last resort (early bean creation)

Code to remember

```
@ConfigurationProperties(prefix = "shop")
@Validated
record ShopProperties(@NotBlank String name,
                      @DefaultValue("30s") Duration checkoutTimeout,
                      @Valid Payment payment) {
    record Payment(@Min(0) int retries) {}
}
// + @ConfigurationPropertiesScan on the application class
```

In the interview

Q: Name the main property sources in priority order. A: Tests > command line > `SPRING_APPLICATION_JSON` > system properties > env vars > `random.*` > profile files (outside > inside) > `application.*` > `@PropertySource` > defaults — "more outside and later is stronger".

Q: How does auto-configuration work? A: Each module lists `@AutoConfiguration` classes in the `.imports` file; Boot filters by classpath, sorts, processes them after your beans, and each bean is guarded by conditions so it fills gaps and backs off.

Q: Why didn't Boot create bean X? A: Run with `--debug` (condition evaluation report: positive/negative matches with reasons) or `/actuator/conditions`; `/actuator/beans` lists what exists.

Q: How do you override an auto-configured bean? A: Declare your own bean of that type, or better, a property or customizer; exclude the whole auto-configuration only when needed.

Q: How do you make the app refuse to start with bad configuration? A: `@ConfigurationProperties` + `@Validated` + constraints; binding failures stop startup.

Q: How do you load Kubernetes secrets? A: `spring.config.import=optional:configtree:/etc/secrets/` or env vars.

Q: What does Boot 4 modularization mean? A: Per-technology auto-configuration modules with their own starters and test starters, so the classpath has only what you use; auto-configuration class packages moved.

Gotchas

- Value in `application.yaml` "ignored" → an env var with the same relaxed name overrides it.
- Expecting `@PropertySource` to beat `application.yaml` → it is near the bottom.
- Secrets in `application.yaml` in Git → env vars, `configtree:`, a secret manager.
- `@ConditionalOnBean` / `OnMissingBean` in *your own* config → order-dependent; reliable only in auto-configurations (they run last).
- `@ConditionalOnClass` on a `@Bean` method returning that library's type → put it on a nested `@Configuration` class.
- Excluding by a Boot 3 class name → packages moved in Boot 4 (e.g. `org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration`).
- `@EnableWebMvc` in a Boot app → switches off Boot's MVC auto-configuration.
- Typo in a key → silently ignored; use metadata/IDE and validation of required fields.

TL;DR

Outside/later property sources win; group settings into validated `@ConfigurationProperties` records; auto-configuration = `.imports` + conditions + `@ConditionalOnMissingBean`, explained by `--debug`; Boot 4 = per-technology modules and starters.

4. AOP and Proxies

AOP vocabulary, `@Aspect` and pointcuts, JDK vs CGLIB proxies, proxy limitations (proved), features built on proxies, Framework 7 `@Retryable` / `@ConcurrencyLimit`, Spring AOP vs AspectJ.

Must-know

- **AOP** — write a cross-cutting concern (logging, timing, transactions, security) once and apply it to many places.
- **Join point** — in Spring AOP only a **method execution on a bean**; **pointcut** — expression selecting join points; **advice** — code run there (`@Before`, `@AfterReturning`, `@AfterThrowing`, `@After`, `@Around`); **aspect** — class bundling them (`@Aspect` **+** `@Component`, `spring-boot-starter-aspectj` in Boot 4).
- `@Around` — only advice that gets a `ProceedingJoinPoint` and decides whether/how the method runs; must `return pjp.proceed()`.
- **Aspect order** — `@Order`: lower value = **outer** layer (first in, last out).
- **Proxy-based** — a `BeanPostProcessor` returns a proxy for every bean matching a pointcut; callers hold the proxy; no bytecode changes.
- **JDK proxy** implements the interfaces (inject by interface only); **CGLIB** is a generated subclass (no `final` classes; `final` / `private` methods not advised). Boot: CGLIB for all (`spring.aop.proxy-target-class=true`); Framework 7 `@Proxyable` chooses per bean.
- **Self-invocation** — `this.method()` goes to the target, not the proxy → no advice. Fixes: move to another bean (best), lazy self-injection/`ObjectProvider`, `TransactionTemplate`, AspectJ weaving.
- **Framework 7 resilience** — `@Retryable` (`maxRetries`, `delay`, `multiplier`, `jitter`, `maxDelay`, `includes` / `excludes`; reactive too) and `@ConcurrencyLimit` (bulkhead, BLOCK/REJECT), need `@EnableResilientMethods` (no auto-configuration); programmatic `RetryTemplate` / `RetryPolicy` in `spring-core`; **no circuit breaker**.
- **Spring AOP vs AspectJ** — runtime proxies on bean methods vs compile/load-time weaving of any join point (constructors, fields, private methods, self-calls).

Key facts & numbers

Proxy-based features (all share the self-invocation limits):

Annotation	Switched on by	Interceptor
`@Transactional`	Boot auto-config	`TransactionInterceptor`
`@Cacheable` / `@CachePut` / `@CacheEvict`	`@EnableCaching`	`CacheInterceptor`
`@Async`	`@EnableAsync`	`AsyncExecutionInterceptor`
`@Retryable`, `@ConcurrencyLimit`	`@EnableResilientMethods`	retry / concurrency-limit post-processors
`@PreAuthorize`, `@PostAuthorize`, `@Secured`	`@EnableMethodSecurity`	authorization method interceptors
`@Validated` + parameter constraints	Boot auto-config	`MethodValidationInterceptor`
`@Observed`	an `ObservedAspect` bean	aspect
Spring Data repositories, HTTP interface clients	Boot	JDK proxies
full `@Configuration`	always	CGLIB subclass

When advice does NOT run: self-invocation · `private` / `final` / `static` methods · calls from the constructor or `@PostConstruct` · objects made with `new` · missing `@Enable…`.

Symptom	Cause → fix
aspect never runs	not a bean / starter missing / pointcut doesn't match
NPE on an injected field in a method	`final` method ran on the CGLIB proxy instance → remove `final`
`BeanNotOfRequiredTypeException`	JDK proxy injected as the concrete class → inject the interface
advised method returns `null`	`@Around` didn't return `proceed()`
slow startup after adding an aspect	pointcut too broad → `within(com.yourapp..*)`
retry happens inside a failing transaction	transaction interceptor outside retry → retry at a higher layer / order advisors

`@Retryable(maxRetries = 3)` = **4 attempts**; last exception rethrown.

Code to remember

```
@Aspect @Component @Order(1)
class TimingAspect {
    @Around("within(com.shop..*) && @annotation(com.shop.Timed)")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long t = System.nanoTime();
        try { return pjp.proceed(); }          // never forget to return it
        finally { log.info("{} took {} µs", pjp.getSignature(), (System.nanoTime() - t) / 1000); }
    }
}
```

In the interview

Q: How does Spring AOP work at runtime? A: A `BeanPostProcessor` checks each bean against all pointcuts and returns a proxy when any match; the proxy runs the advice chain, then calls the real object.

Q: What is the self-invocation problem? A: `this.saveBatch()` goes to the real object, so `@Transactional`, `@Cacheable`, `@Async`, `@Retryable` on it are silently skipped; move the method to another bean.

Q: JDK proxy vs CGLIB — which does Boot use? A: CGLIB for everything by default, so you can inject by class; JDK proxies implement interfaces only.

Q: How do you see if a bean is proxied? A: `AopUtils.isAopProxy(bean)` / `isCglibProxy`, or `bean.getClass()` containing `$$SpringCGLIB$$` or `$Proxy`.

Q: How do you order two aspects? A: `@Order` — lower value is outer; e.g. retry outside the transaction so each attempt gets a fresh transaction.

Q: What are `@Retryable` and `@ConcurrencyLimit` in Framework 7? A: Core retry with back-off/jitter and a bulkhead, enabled by `@EnableResilientMethods`; no circuit breaker — use Resilience4j.

Q: Danger of retries? A: Duplicate side effects on non-idempotent operations and retry storms; retry only transient errors, add jitter, retry in one layer.

Gotchas

- Aspect without `@Component` → nothing happens, no error.
- Swallowing exceptions in `@Around` → outer `@Transactional` won't roll back.
- `com.shop.*` matches only that package → `com.shop..*` for sub-packages; `(*)` = one parameter, `(..)` = any.
- Custom trigger annotation only on the interface method → put it on the implementation.
- Forgetting `@EnableResilientMethods` → no retries, no error; no jitter on a fleet → synchronised retry waves.

TL;DR

Advice runs only through the proxy: self-invocation, `private`/`final`, constructor/`@PostConstruct` calls and `new` objects skip `@Transactional`, `@Cacheable`, `@Async`, `@Retryable`, `@PreAuthorize`. Boot proxies with CGLIB; Framework 7 adds `@Retryable`/`@ConcurrencyLimit` (no circuit breaker).

5. Spring MVC and REST APIs

The `DispatcherServlet` flow, controllers and mappings, Jackson 3, validation, Problem Details, content negotiation, Framework 7 API versioning, CORS, uploads, filters vs interceptors, springdoc.

Must-know

- **`@RestController`** = `@Controller` + `@ResponseBody` — return values are written to the body, not treated as view names.
- **Inputs** — `@PathVariable` (identifies a resource), `@RequestParam` (query: filters/paging; `required = false`/default), `@RequestBody` (deserialized JSON of POST/PUT/PATCH), `@RequestHeader`.
- **`ResponseEntity`** — control status and headers: `201` + `Location` (`ResponseEntity.created(uri)`), `204`, `304` + `ETag`.
- **Validation** — constraints on the DTO + `@Valid` on `@RequestBody` → `MethodArgumentNotValidException` → 400; parameter constraints (`@RequestParam @Min(1) int page`) validated built-in since 6.1 → `HandlerMethodValidationException`; groups via `@Validated(Group.class)`; needs `spring-boot-starter-validation`.
- **Problem Details (RFC 9457, obsoletes 7807)** — `application/problem+json` with `type`, `title`, `status`, `detail`, `instance` + extensions; one `@RestControllerAdvice extends ResponseEntityExceptionHandler`.
- **Jackson 3** — `tools.jackson.*` (annotations stay `com.fasterxml.jackson.annotation`), immutable `JsonMapper`, unchecked `JacksonException`, java.time built in, ISO dates; customize with `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`; properties `spring.jackson.json.*`.
- **Content negotiation** — `Accept` + `produces` pick the `HttpMessageConverter`; none → **406**; unsupported `Content-Type` → **415**.
- **API versioning (Framework 7)** — `spring.mvc.apiversion.*` (header, query param, path segment or media-type parameter), default + supported versions, `@GetMapping(path = "/orders", version = "1.1+")`; unsupported version → 400; deprecation handler adds `Deprecation`, `Sunset`, `Link` headers.
- **CORS** — browser rule; preflight `OPTIONS` for non-simple requests; `addCorsMappings`/`@CrossOrigin` in MVC, and with Security a `CorsConfigurationSource` + `http.cors(...)`. Does not protect from non-browser clients.
- **springdoc 3.x** for OpenAPI on Boot 4 (`/v3/api-docs`, Swagger UI).

Key facts & numbers

Request flow:

#	Step	What happens
1	Servlet filters (incl. Spring Security)	every request; can wrap request/response
2	`DispatcherServlet`	front controller
3	`HandlerMapping`	picks method by path, HTTP method, consumes/produces, **API version**
4	Interceptors `preHandle`	know the `HandlerMethod`
5	`HandlerAdapter`	resolves arguments: converters for bodies, conversion, validation
6	Controller method	your code
7	`HttpMessageConverter`	writes the return value per `Accept`
8	`HandlerExceptionResolver`s	`@ExceptionHandler` / `@ControllerAdvice` on exceptions
9	`afterCompletion` → filters	back out

Error-handling lookup order: `@ExceptionHandler` in the controller → in `@ControllerAdvice` beans → `@ResponseStatus`/`ResponseStatusException` → Spring defaults → Boot's `/error`.

Option	Use when
`spring.mvc.problemdetails.enabled=true` (default `false`)	no advice of your own; Problem Details for Spring's exceptions
`@ResponseStatus` on an exception class	quick, no custom body
`ResponseStatusException`	one-off errors
extend `ErrorResponseException`	exception carries its own `ProblemDetail`
`@RestControllerAdvice(basePackages/assignableTypes)`	different handlers per API area

Method	Safe	Idempotent	Response
GET	yes	yes	200 / 404
POST (create)	no	**no**	201 + `Location`
PUT (replace)	no	yes	200 / 204
PATCH (partial)	no	not necessarily	200 / 204
DELETE	no	yes	204, then 404

	Filter	`HandlerInterceptor`	Aspect
Runs	every request (static, errors too)	only mapped requests, inside `DispatcherServlet`	bean method calls
Knows the handler	no	yes	yes (join point)
Wrap request/response	yes	no	no
Use	security, request ids/MDC, compression	per-endpoint checks, locale, timing	business cross-cutting

Code to remember

```
@RestControllerAdvice
class ApiErrors extends ResponseEntityExceptionHandler {
    @ExceptionHandler(BookNotFoundException.class)
    ProblemDetail notFound(BookNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, "Book not found");
        pd.setType(URI.create("https://api.example.com/problems/book-not-found"));
        pd.setProperty("bookId", ex.id());
        return pd;
    }
}
```

In the interview

Q: Walk me through a request in Spring MVC. A: Filters → `DispatcherServlet` → `HandlerMapping` → interceptors → `HandlerAdapter` resolves args (converters, validation) → method → converter by `Accept` → exception resolvers → `afterCompletion` → filters.

Q: How do you handle errors consistently? A: One `@RestControllerAdvice` extending `ResponseEntityExceptionHandler` mapping domain exceptions to statuses and returning `ProblemDetail`; unexpected ones → 500, logged, no stack traces.

Q: How do you validate a request? A: Jakarta constraints on the DTO + `@Valid` on the body (400 via `MethodArgumentNotValidException`); parameter constraints are validated built-in since 6.1.

Q: How do you version an API in Boot 4? A: Framework 7 API versioning: `spring.mvc.apiversion.use.header=X-API-Version`, supported/default versions, `version = "1.1+"` on mappings, deprecation headers; clients use `ApiVersionInserter`.

Q: What changed with JSON in Boot 4? A: Jackson 3 — new packages, immutable `JsonMapper`, unchecked exceptions, ISO dates by default; `JsonMapperBuilderCustomizer`, `@JacksonComponent`.

Q: Filter vs interceptor? A: Filter: servlet level, every request, can wrap objects, doesn't know the handler; interceptor: MVC level, knows the `HandlerMethod`, only mapped requests.

Q: How do you paginate? A: `page`/`size`/`sort` via `Pageable`, cap `size`, return a stable page DTO; keyset (cursor) pagination for huge/changing data. Idempotent retries of POST: accept an `Idempotency-Key`.

Gotchas

- Missing `@Valid` (or `@Valid` on nested fields, or the validation starter) → constraints silently ignored; `@NotNull` on a primitive never fires.
- `@Validated` on a controller class → `ConstraintViolationException` → 500; rely on built-in method validation.
- Returning `200` with an `"error"` field / a catch-all `Exception` → 400 → bugs are 500s; don't leak `ex.getMessage()` of SQL errors.
- `/{id}` catching `/export` → constrain `{id:\d+}`; trailing slash no longer matched → 404.
- Jackson 2 `ObjectMapper` bean in Boot 4 → not used for HTTP (MVC uses the Jackson 3 mapper).
- Exposing JPA entities as JSON → lazy errors, recursion, leaked columns; use DTO records.
- Header set in `postHandle` missing → body already written; use `preHandle` or a filter.
- `allowedOrigins("*")` + `allowCredentials(true)` → rejected; CORS only in MVC with Security present → preflight gets 401.
- `@SpringBootTest` can't autowire `MockMvc` in Boot 4 → add `@AutoConfigureMockMvc`.

TL;DR

Filters → `DispatcherServlet` → mapping → interceptors → args + validation → controller → converter → exception resolvers. One advice returning RFC 9457 `ProblemDetail`; Jackson 3 (`tools.jackson`); Framework 7 `version = "1.1+"` API versioning.

6. REST Clients and HTTP Interfaces

Choosing a client, `RestClient`, error handling, timeouts, interceptors, `@HttpExchange` interfaces and Boot 4 client groups, client-side versioning and retries, testing clients.

Must-know

- **`RestClient`** (Framework 6.1) — fluent, synchronous; in Boot 4 add `spring-boot-starter-restclient` and inject the **prototype** **`RestClient.Builder`** (Boot's converters, HTTP library, timeouts, observability); build once, reuse — it is thread-safe.
- **Calls** — `get().uri(template, vars).retrieve().body(T)` / `toEntity(T)` / `toBodilessEntity()`; `ParameterizedTypeReference` for generics; `exchange()` = raw control, **status handlers not applied**.
- **Errors** — 4xx → `HttpClientErrorException.*`, 5xx → `HttpServerErrorException.*`, I/O/timeouts → `ResourceAccessException`; translate at the client edge with `onStatus`/`defaultStatusHandler`; `getResponseBodyAs(ProblemDetail.class)`.
- **Timeouts** — `spring.http.clients.connect-timeout` / `read-timeout` (Boot 4 names; were `spring.http.client.*`), or per group; no timeout ≈ wait forever.
- **HTTP service interfaces** — an interface with `@GetExchange`/`@PostExchange`; `HttpServiceProxyFactory` makes a **JDK proxy**. Boot 4: `@ImportHttpServices(group = "catalog", types = CatalogApi.class)` + `spring.http.serviceclient.<group>.*` (base-url, timeouts, default headers, apiversion); customize with `RestClientHttpServiceGroupConfigurer`.

- **Client-side versioning** (Framework 7) — `ApiVersionInserter.useHeader(...)` + `defaultApiVersion(...)`, per request `.apiVersion(...)`, `@GetExchange(version = ...)`.
- **Retries** — only idempotent + transient failures, exponential back-off with jitter, total budget, one layer; add a circuit breaker.

Key facts & numbers

REST client choice:

Client	Style	Since / status	Use for
`RestClient`	fluent, blocking	F6.1; recommended blocking client	hand-written calls in MVC/non-reactive apps
HTTP interface `@HttpExchange`	declarative interface → proxy	F6.0; Boot 4 auto-config (`@ImportHttpServices`)	typed clients to other services, least boilerplate
`WebClient`	non-blocking `Mono`/`Flux`	F5.0; recommended reactive client (`spring-boot-starter-webclient`)	WebFlux, streaming
`RestTemplate`	template methods	Spring 3.0; `@Deprecated` planned in 7.1 (Nov 2026), removal in 8.0	existing code only → `RestClient.create(restTemplate)`

Underlying HTTP library, auto-detected in order: Apache HttpClient 5 → Jetty → Reactor Netty → JDK `HttpClient` → `HttpURLConnection`.

Mistake	Fix
new `RestClient` per request	build once, reuse (pooling)
`RestClient.create()` in a Boot app	inject `RestClient.Builder`
no timeouts	`spring.http.clients.*-timeout` / per group
string-concatenated URLs	URI templates / `UriBuilder`
retrying POSTs or 4xx	retry only idempotent + transient
`HttpClientErrorException` leaking into business code	translate to domain exceptions
mock-only tests	also test against a real/stub server

Code to remember

```
@HttpExchange("/api/products")
interface CatalogApi {
    @GetExchange("/{id}") Product get(@PathVariable long id);
}
@ImportHttpServices(group = "catalog", types = CatalogApi.class)
@Configuration class ClientsConfig {}
// spring.http.serviceclient.catalog.base-url=http://catalog:8080
// spring.http.serviceclient.catalog.read-timeout=2s
```

In the interview

Q: Which HTTP client for a new Boot 4 MVC service? A: `RestClient`, or an `@HttpExchange` interface backed by it — Boot configures converters, timeouts, observability; `WebClient` is for reactive; with virtual threads blocking calls are cheap.

Q: How are HTTP interfaces implemented? A: `HttpServiceProxyFactory` creates a JDK proxy; Boot 4's `@ImportHttpServices` registers groups configured by `spring.http.serviceclient.<group>.*`.

Q: How do you handle errors and timeouts? A: Map `HttpClientErrorException`/`HttpServerErrorException`/`ResourceAccessException` to domain exceptions with `onStatus`; always set connect and read timeouts sized from downstream p99 and your SLA.

Q: How do you make remote calls resilient? A: Timeouts, retries with back-off + jitter for idempotent/transient failures (`@Retryable`), circuit breaker + bulkhead (Resilience4j/`@ConcurrencyLimit`), fallbacks, idempotency keys for writes.

Q: How do you test a REST client? A: `@RestClientTest` + `MockRestServiceServer` for request/response mapping, plus a test against a real stub server (WireMock) for HTTP-level issues.

Gotchas

- Catching `Exception` and returning `null` → hides outages; translate and log.
- Treating every 4xx as retryable → a 400 fails the same way every time.
- Logging full error bodies → may contain personal data or secrets.
- `WebClient.Builder` missing in Boot 4 → add `spring-boot-starter-webclient`.

TL;DR

`RestClient` from the injected builder (or `@HttpExchange` + `@ImportHttpServices`) for new code; always set timeouts; translate HTTP exceptions at the edge; retry only idempotent transient failures; `RestTemplate` is on its way out (7.1 deprecation, 8.0 removal).

7. Data Access with JDBC and Transactions

`DataSource` and HikariCP, `JdbcClient`, exception translation, how `@Transactional` works, all seven propagations, rollback rules, isolation, pitfalls, `TransactionTemplate`, `@TransactionalEventListener` — proved on real PostgreSQL.

Must-know

- **Spring over JDBC** — pooled `DataSource` (HikariCP), `JdbcClient`/`JdbcTemplate` (connections, statements, mapping, cleanup), `SQLException` → unchecked **`DataAccessException`** hierarchy (`DuplicateKeyException`, `DataIntegrityViolationException`; the *transient* branch = "retry may work"), declarative transactions.
- **`JdbcClient`** (6.1) — `sql(...).param(...).query(Order.class).list()/optional()/single()`, `update()`; `JdbcTemplate.batchUpdate` for batches. Always bind parameters.
- **Pool size** — small; HikariCP default **10**; pool × instances < database `max_connections`; watch `hikaricp.connections.pending`. Virtual threads still share the same few connections.
- **How `@Transactional` works** — proxy → `TransactionInterceptor` → `PlatformTransactionManager` (`JdbcTransactionManager`, `JpaTransactionManager`) takes a connection, auto-commit off, **binds it to the current thread**; repositories on that thread reuse it; return → commit, throw → rollback rules.
- **Defaults** — propagation `REQUIRED`, isolation `DEFAULT` (= database: READ COMMITTED on PostgreSQL/Oracle/SQL Server, REPEATABLE READ on MySQL InnoDB), `readOnly = false`, `timeout = -1`, rollback on `RuntimeException`/`Error` only.
- **Rollback-only / `UnexpectedRollbackException`** — an inner `REQUIRED` method threw → shared transaction marked rollback-only → outer caught it and returned → commit attempt throws `UnexpectedRollbackException`.
- **`readOnly = true`** — read-only JDBC connection (PostgreSQL rejects writes), Hibernate skips dirty checking/flush, a routing `DataSource` can use a replica. Not a security feature.
- **Isolation/timeout apply only where the transaction starts** — ignored on a method that joins.
- **Where** — on the service method that is one business operation; not controllers (too wide), not only repositories (too narrow); no remote HTTP calls inside.
- **`TransactionTemplate`** — `txTemplate.execute(status -> …)`: boundary smaller than a method, decided at runtime, no proxy pitfalls.
- **`@TransactionalEventListener`** — default `AFTER_COMMIT`; skipped without a transaction unless `fallbackExecution = true`; if it writes, it must be `REQUIRES_NEW` (or `NOT_SUPPORTED`); not guaranteed delivery → outbox.

Key facts & numbers

Propagation (all seven proved):

Propagation	A transaction exists	None exists	Typical use
`REQUIRED` (default)	**join** (same connection)	start one	almost everything
`REQUIRES_NEW`	**suspend**, start independent one on a **second connection**	start one	audit log that must survive a rollback
`NESTED`	**savepoint**; inner failure rolls back to it only	start one	"try this part" (JDBC; avoid with JPA)
`MANDATORY`	join	**throw** `IllegalTransactionStateException`	must be called in a transaction
`SUPPORTS`	join	run without	read helpers
`NOT_SUPPORTED`	suspend, run without	run without	slow non-transactional work
`NEVER`	**throw**	run without	guard

Rollback rules (proved):

Thrown out of the method	Default	`rollbackFor = X`	`noRollbackFor = X`	`rollbackOn = ALL_EXCEPTIONS` (F6.2)
`RuntimeException` / `Error`	**rollback**	rollback	commit if X	rollback
checked `Exception`	**commit (!)**	rollback if X	commit	rollback
caught inside the method	commit — proxy never sees it	commit	commit	commit

Global: `@EnableTransactionManagement(rollbackOn = RollbackOn.ALL_EXCEPTIONS)` (reference docs recommend it). Mark without throwing: `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`.

Isolation	Prevents	Notes
`READ_COMMITTED`	dirty reads	non-repeatable reads, lost updates still possible
`REPEATABLE_READ`	+ non-repeatable reads	PostgreSQL: one snapshot; concurrent update → `40001` → retry
`SERIALIZABLE`	+ phantoms, write skew	expect serialization failures; always retry

Pitfalls — "why is my `@Transactional` not working?"

Symptom	Cause
partial writes after an exception	self-invocation, `private` method, bean made with `new`
NPE inside a `final` `@Transactional` method	ran on the CGLIB proxy instance (fields `null`)
committed although it threw	checked exception without `rollbackFor`, or caught inside
`UnexpectedRollbackException`	inner `REQUIRED` failure was caught
isolation/timeout ignored	method joined an existing transaction
works in tests, fails in prod	the test was `@Transactional` and supplied the transaction
`@Async` code doesn't see the data	transactions are thread-bound
pool exhausted under load	`REQUIRES_NEW` holds two connections; remote calls inside transactions
wrong manager	writes go through another `DataSource` → `@Transactional("ordersTx")`

Code to remember

```
@Service
class TransferService {
    private final JdbcClient jdbc;
    TransferService(JdbcClient jdbc) { this.jdbc = jdbc; }

    @Transactional(rollbackFor = InsufficientFundsException.class)
    public void transfer(long from, long to, long cents) throws InsufficientFundsException {
        jdbc.sql("UPDATE account SET balance = balance - :c WHERE id = :id")
            .param("c", cents).param("id", from).update();
        // ... credit 'to'; throw InsufficientFundsException → rollback
    }
}
```

In the interview

Q: How does `@Transactional` work internally? A: A proxy's `TransactionInterceptor` asks the transaction manager to begin (connection bound to the thread), calls the method, then commits or applies rollback rules.

Q: Checked exception — commit or rollback? A: Commit (EJB convention); use `rollbackFor` or `rollbackOn = ALL_EXCEPTIONS`; a caught exception never rolls back.

Q: `REQUIRES_NEW` vs `NESTED`? A: A separate physical transaction on a second connection vs a savepoint in the same transaction (JDBC; Hibernate's persistence context doesn't roll back to savepoints).

Q: Why might `@Transactional` be silently ignored? A: Self-invocation, `private`/`final` method, not a bean, exception caught, checked exception, different transaction manager, work on another thread.

Q: How big should the pool be? A: Small (default 10); the database runs only as many queries as cores/disks allow; size from measurements and pool × instances.

Q: How do you handle a serialization failure or deadlock? A: They surface as `ConcurrencyFailureException` subtypes; retry the **whole** transaction from outside it with back-off.

Q: Does `@Transactional` work with `@Async`? A: The async method runs on another thread without the caller's transaction; it can have its own, separate one.

Q: How do you run code only after commit? A: `@TransactionalEventListener` (AFTER_COMMIT) or `TransactionSynchronization.afterCommit`; for guaranteed delivery, an outbox.

Gotchas

- `REQUIRES_NEW` everywhere "for safety" → doubles connection use, breaks atomicity.
- Catch + log + no rethrow → **commits**; wrapping a runtime exception into a checked one also flips to commit.
- `maximum-pool-size=100` "to be safe" → out of DB connections when you scale out; `connection-timeout` 30 s default → fail faster in user-facing APIs.
- SERIALIZABLE without a retry loop; testing isolation on H2 (behaves differently from PostgreSQL).
- String concatenation in SQL → injection; `.single()` when "no row" is normal → `.optional()`.
- AFTER_COMMIT listener writing without `REQUIRES_NEW` → fails at startup (since 6.1).

TL;DR

`@Transactional` = proxy + transaction manager + thread-bound connection. REQUIRED joins, REQUIRES_NEW = second connection, NESTED = savepoint. Checked exceptions commit; caught exceptions never roll back; a joined failure dooms the whole transaction.

8. Spring Data JPA and Hibernate

Entities and ids, the persistence context, relationships, fetch types, N+1 measured and fixed, repositories and queries, pagination, projections, auditing, locking, open-in-view, Flyway.

Must-know

- **JPA** (Jakarta Persistence 3.2) = spec; **Hibernate 7** (7.4 in Boot 4.1) = implementation; **Spring Data JPA** (2026.0) = generated repositories, derived queries, paging, auditing.
- **Persistence context** — the `EntityManager`'s first-level cache and unit of work: one managed instance per row + a snapshot; at flush **dirty checking** issues `UPDATE`s — no `save()` needed after changing a loaded entity inside a transaction.

- **Entity states** — transient (new), managed (tracked), detached (context closed, not tracked), removed (deleted at flush).
- **`save()`** — `persist` if new (null id, or null wrapper `@Version`), else `merge` (may `SELECT` first); assigned ids → implement `Persistable.isNew()` or use a version.
- **Owning side** — the side holding the FK (`@ManyToOne`); JPA writes only from it; inverse side `mappedBy`; sync both in a helper (`addItem`).
- **Cascade REMOVE vs `orphanRemoval`** — deleting the parent deletes children vs removing a child from the collection deletes it; only for true composition, never `@ManyToMany`.
- **Fetch defaults** — to-one EAGER, to-many LAZY → make everything LAZY, fetch per use case.
- **`LazyInitializationException`** — touching an uninitialized lazy proxy after its persistence context closed (controller, JSON). Fix: fetch in the query inside the transaction, return DTOs — not EAGER, not OSIV.
- **N+1** — 1 query for N parents + 1 per parent's lazy association (book demo: 5 statements for 4 authors).
- **Open Session in View** — `spring.jpa.open-in-view` default **`true`** (with a startup warning): keeps the context open through rendering; hides N+1, holds connections → turn off for APIs.
- **Optimistic locking** — `@Version`: `UPDATE … WHERE id = ? AND version = ?`; 0 rows → `ObjectOptimisticLockingFailureException` → 409 or retry. **Pessimistic**: `@Lock(PESSIMISTIC_WRITE)` → `SELECT … FOR UPDATE` for short, contended updates.
- **Schema** — Flyway/Liquibase own it; `ddl-auto=validate`/`none`; Boot 4 needs `spring-boot-starter-flyway` (`flyway-core` alone isn't auto-configured).
- **`equals`/`hashCode`** — not Lombok `@Data`; natural key, or id-based with a constant `hashCode`.

Key facts & numbers

JPA fetch / N+1 fixes:

Fix	Queries	Good for	Watch out
`JOIN FETCH` (JPQL)	1	one known use case	inner join by default; two `List` collections → `MultipleBagFetchException`
`@EntityGraph(attributePaths = "books")`	1	same, declared on a repository method (left join)	cartesian product with several collections
`default_batch_fetch_size` / `@BatchSize`	1 + N/size	global safety net (e.g. 16)	still >1 query
DTO projection	1	read-only screens/APIs	no entities → no dirty checking
EAGER	—	**never a fix**	makes it worse

Paging + `JOIN FETCH` of a collection: Hibernate 7.4 pages parents in a subquery (2 statements); older versions paged **in memory** (`HHH90003004`) — set `hibernate.query.fail_on_pagination_over_collection_fetch=true` there.

Query style	Use
derived (`findByStatusAndCreatedAtAfter`)	short conditions
`@Query` JPQL	long names, joins, fetches (entity names, not tables)
`nativeQuery = true`	window functions, JSONB
`@Modifying` (+ `clearAutomatically = true`)	bulk update/delete
`Page` / `Slice` / `Window`	extra COUNT + totals / `size + 1` "has next" / keyset scrolling
projections	interface (proxy), record DTO, dynamic `<T> findBy(..., Class<T>)`

Performance checklist: all to-one LAZY · statement counts asserted in tests · `hibernate.default_batch_fetch_size=16` · read-only DTOs + `readOnly = true` · batched inserts need SEQUENCE ids + `hibernate.jdbc.batch_size=50` + `order_inserts` (IDENTITY disables insert batching) · OSIV off · paging everywhere · big jobs chunk + `flush()`/`clear()` · index foreign keys (PostgreSQL doesn't).

Code to remember

```
interface AuthorRepository extends JpaRepository<Author, Long> {
    @Query("select distinct a from Author a left join fetch a.books where a.country = :c")
    List<Author> withBooks(@Param("c") String country);      // 1 query instead of 1 + N

    @EntityGraph(attributePaths = "books")
    List<Author> findByCountry(String country);
}
```

In the interview

Q: What is the persistence context / dirty checking? A: The `EntityManager`'s unit of work holding one managed instance per row with a snapshot; at flush Hibernate compares and issues `UPDATE`s for changed entities.

Q: Explain N+1 and fixes. A: 1 + N round trips from lazy loading per row; `JOIN FETCH`, `@EntityGraph`, batch fetching, DTO projections; detect by counting statements in tests.

Q: `LazyInitializationException`? A: Lazy association touched after the transaction ended; fetch what the use case needs in the query and return DTOs.

Q: JOIN FETCH vs `@EntityGraph`? A: Same idea — JPQL inner join (unless `left`) vs declared on the method, left join.

Q: How does optimistic locking work? A: `@Version` checked in the `UPDATE`'s WHERE clause; 0 rows → `ObjectOptimisticLockingFailureException`; return 409 or retry the read-modify-write.

Q: Should OSIV be on? A: No for APIs — it hides N+1, holds connections longer and moves DB access out of the service layer.

Q: `Page` vs `Slice`? A: `Page` runs an extra COUNT; `Slice` fetches `size + 1` to know if there's a next — cheaper for infinite scroll.

Gotchas

- Setting only the inverse side → nothing saved; `@OneToMany` without `mappedBy` → extra join table.
- `cascade = ALL` everywhere → deletes shared data.
- Returning entities from controllers / `toString()` with associations → lazy loads, N+1, schema leak.
- Testing with 3 rows → N+1 invisible; assert statement counts.
- 100 000 entities in one transaction → context + snapshots in memory; page + `clear()`, or projections/JDBC.
- Bulk `@Modifying` update then reading stale managed entities → `clearAutomatically = true`.
- Modifying the old detached object after `merge()` → use the returned instance.

TL;DR

Persistence context + dirty checking; everything LAZY, fetch per use case (`JOIN FETCH`, `@EntityGraph`, batch size, DTOs); OSIV off; `@Version` for concurrent edits; Flyway owns the schema (`spring-boot-starter-flyway`, `ddl-auto=validate`).

9. Spring Data beyond JPA, and Caching

The Spring Data family, Data JDBC aggregates, Data REST, Flyway vs Liquibase, MongoDB and Redis, the caching abstraction and its pitfalls, choosing a data technology.

Must-know

- **Spring Data** — one repository model for every store; only mapping and store features differ.
- **Spring Data JDBC** — maps **aggregates** (root + owned children, one repository per root) with plain SQL: no persistence context, no lazy loading, no dirty checking; loading is eager; `save` writes immediately and **deletes + re-inserts children**; other aggregates referenced by id (`AggregateReference`); `@Version` on the root protects the whole aggregate.
- **"New or existing?"** — new if id is `null` (or `0`) or `@Version` is `null` / `0`. An **assigned id** without `@Version` → `save` runs an `UPDATE` of **0 rows, no exception** — data silently lost. Fix: `@Version`, `Persistable.isNew()`, `JdbcAggregateTemplate.insert()`, or DB-generated ids.
- **Spring Data REST** — HAL CRUD endpoints from repositories; exports every public repository by default (`exported = false` / `detection-strategy=annotated`); secure it.
- **Flyway** — versioned SQL/Java + repeatable migrations with checksums (an edited applied migration stops startup). **Liquibase** — changelogs, rollbacks, preconditions, many databases; FSL-licensed since 5.0.

- **MongoDB** — Boot 4 connection properties `spring.mongodb.*` (were `spring.data.mongodb.*`); no auto-configured transactions — declare `MongoTransactionManager` + replica set. **Redis** — templates for data structures, `@RedisHash` repositories (equality queries via index sets).
- **Cache abstraction** — `@EnableCaching` + proxy `CacheInterceptor`: `@Cacheable` (skip method on hit), `@CachePut` (always run, store), `@CacheEvict` (`allEntries = true`); pluggable `CacheManager` (Boot detects Redis **before** Caffeine; fallback `ConcurrentHashMap` "simple").
- **Default key** — `SimpleKeyGenerator`: no args → `SimpleKey.EMPTY`, one arg → the arg, several → `SimpleKey` — **without the method name**; SpEL `key = "#id"`; args need `equals`/`hashCode`.
- **`condition` vs `unless`** — before the call on arguments vs after on `#result` (`unless = "#result == null"`).
- **`sync = true`** — one caller computes per key (stampede protection) **per JVM**; can't combine with `unless`.

Key facts & numbers

Caching pitfalls:

Pitfall	Symptom	Fix
no `@EnableCaching` / self-invocation / non-public method	cache never consulted, no error	enable; move method to another bean
cached **mutable** object (Caffeine stores references)	one caller's change corrupts it for all	cache immutable values (records, `List.copyOf`)
`@CachePut`/`@CacheEvict` inside a rolled-back transaction	cache holds uncommitted data	`TransactionAwareCacheManagerProxy`, Redis `transactionAware()`, or evict AFTER_COMMIT
local cache in a cluster	other instances stale until TTL	short TTL, Redis, broadcast evictions, two-level cache
no size bound / `simple` provider in prod	`OutOfMemoryError`	`maximumSize`, real provider
per-user data under a shared key	users see each other's data	include user/tenant in key
hot key expires under load	stampede (8 concurrent misses → 8 loads)	`sync = true` (→ 1 load, per JVM)
class shape changed with Redis	old entries fail to deserialize	versioned key prefix (`shop:v3:`), short TTL
hit rate unknown	—	`cache.gets` metrics (Caffeine needs `recordStats()`)

	Caffeine (local)	Redis (shared)
Read cost	nanoseconds, no serialization	network hop + (de)serialization
Across instances	own copy each (stale until TTL)	one copy
Stores	object references	serialized copies
TTL / size	`spring.cache.caffeine.spec` (`expireAfterWrite`, `maximumSize`)	`spring.cache.redis.time-to-live`
Choose for	small, hot, rarely changing reference data	shared, sessions, expensive results

Need	Choose
rich object graph, team knows JPA	Spring Data JPA
clear aggregates, predictable SQL	Spring Data JDBC
full SQL control, reports, bulk	`JdbcClient` (or jOOQ)
non-blocking relational	R2DBC
nested documents read whole	MongoDB
counters, sessions, rate limits, shared cache	Redis

In the interview

Q: Spring Data JDBC vs JPA? A: JDBC: no persistence context, lazy loading or dirty checking; eager aggregates, children rewritten on save, predictable SQL. JPA: caching, lazy loading, rich mapping, more hidden behaviour.

Q: Why did saving a `Product` with an assigned id insert nothing? A: Non-null id and no `@Version` means "existing" → `UPDATE` of 0 rows; add `@Version`, `Persistable` or call `insert()`.

Q: How does `@Cacheable` work internally? A: Proxy + `CacheInterceptor` → key → `CacheManager` → `Cache` lookup; the method runs only on a miss.

Q: Caffeine or Redis? A: Caffeine fastest but per-instance and stale across instances; Redis shared and survives restarts at the cost of a network hop and serialization.

Q: What cache problems do you plan for? A: Staleness (TTL, evict on write), stampede (`sync = true`), unbounded growth (max size), mutable values, per-user keys, measuring hit rate.

Q: Flyway vs Liquibase? A: SQL-first, one database → Flyway; rollbacks, preconditions, many databases → Liquibase.

Gotchas

- Spring Data JDBC: expecting dirty checking → changes lost unless you `save`; huge child collections → thousands of deletes/inserts per save.
- Mapping another aggregate as an object reference → it gets deleted/re-inserted with yours; use ids.
- Spring Data REST: `DELETE /api/customers/1` works for anyone until Security rules exist.
- Code that worked with Caffeine behaves differently on Redis (references vs serialized copies).
- Flaky tests from cached values → clear caches in `@BeforeEach` or `spring.cache.type=none`.

TL;DR

Data JDBC = aggregates, no magic, assigned ids need `@Version`. Caching is proxy-based: default key has no method name, cache immutable values, evict after commit, bound size, `sync = true` only per JVM; Caffeine local, Redis shared.

10. Spring Security 1: Fundamentals

The filter chain, the Security 7 lambda DSL, authentication, password hashing, authorization and 401 vs 403, method security, CSRF, CORS, sessions, headers, `spring-security-test`.

Must-know

- **Authentication** = who are you; **authorization** = may you; linked by the `Authentication` object (principal + authorities).
- **Starter defaults** — every URL requires login; form login + HTTP Basic; one user `user` with a random password in the log ("Using generated security password"); CSRF, security headers, session-fixation protection on. Your own `SecurityFilterChain` (or `UserDetailsService`) makes Boot's back off.
- **Architecture** — container → `DelegatingFilterProxy` → `FilterChainProxy` → **first** `SecurityFilterChain` whose `securityMatcher` matches → ordered filters → `AuthorizationFilter` last. Several chains: `securityMatcher` + `@Order`, specific first, catch-all last.
- **Security 7 DSL** — `SecurityFilterChain` `@Bean`, **lambda DSL only** (one lambda per feature); `and()`, `authorizeRequests` and Ant/MVC matchers removed (`PathPatternRequestMatcher` behind `requestMatchers("/x/**")`); `WebSecurityConfigurerAdapter` gone since 6.0.
- **Authentication flow** — filter builds an unauthenticated token → `AuthenticationManager` (`ProviderManager`) → `DaoAuthenticationProvider` → `UserDetailsService` + `PasswordEncoder` → authenticated `Authentication` into `SecurityContextHolder` (**`ThreadLocal`**), and the HTTP session for form login. `UsernameNotFoundException` → `BadCredentialsException` (no enumeration).
- **Passwords** — `PasswordEncoderFactories.createDelegatingPasswordEncoder()`: bcrypt default, Argon2id/scrypt/PBKDF2; `{bcrypt}` prefix; upgrade on login via `UserDetailsPasswordService`. Salt = not secret; pepper = optional secret outside the DB.
- **Role vs authority** — authority = any permission string (`orders:write`, `SCOPE_orders.read`); role = authority with `ROLE_` prefix; `hasRole("ADMIN")` checks `ROLE_ADMIN`. `RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER")`.
- **Rules** — checked top-down, **first match wins**: specific first, end with `anyRequest().authenticated()` or `denyAll()`.
- **Method security** — `@EnableMethodSecurity` + `@PreAuthorize`/`@PostAuthorize`/`@PreFilter`/`@PostFilter`; proxy-based; ownership rules URL patterns can't express; denial → 403.
- **CSRF** — token required on POST/PUT/PATCH/DELETE; keep it whenever the browser sends credentials automatically (cookies); disable only on header-token (bearer) chains; `csrf.spa()` for cookie SPAs.
- **CORS** — `CorsConfigurationSource` bean + `http.cors(...)` so preflights are answered before authentication; exact origins.

- **Sessions** — session id changes at login (fixation defence); logout is `POST /logout` + CSRF token; `STATELESS` never creates sessions. **Headers** — five protective headers by default (HSTS only over HTTPS); add a CSP yourself.
- **Security 7 new** — built-in MFA (`FACTOR_*` authorities), `csrf.spa()`, Password4j encoders, PKCE by default for OAuth2 clients, password grant removed, Authorization Server merged into Spring Security.

Key facts & numbers

Security filter chain (order):

Filter	Job
`SecurityContextHolderFilter`	load `SecurityContext` (e.g. from session), clear afterwards
`HeaderWriterFilter`	security headers
`CorsFilter`	answer preflights (if `cors()` configured)
`CsrfFilter`	reject unsafe requests without a valid token
`LogoutFilter`	`POST /logout`
`UsernamePasswordAuthenticationFilter` / `BasicAuthenticationFilter` / `BearerTokenAuthenticationFilter`	form / Basic / bearer authentication
`AnonymousAuthenticationFilter`	anonymous `Authentication` if nobody authenticated
`ExceptionTranslationFilter`	security exceptions → 401/redirect or 403
`AuthorizationFilter`	last gate: `AuthorizationManager` decides

401 vs 403:

	401 Unauthorized	403 Forbidden
Means	not authenticated — "who are you?"	known but not allowed — "no"
Decided by	`ExceptionTranslationFilter`: denied + **anonymous** → `AuthenticationEntryPoint`	denied + **authenticated** → `AccessDeniedHandler`
Response	`WWW-Authenticate` challenge (APIs) or 302 to `/login` (browsers)	403
Typical causes	missing/expired/invalid token, wrong issuer/audience, CORS preflight hitting security	`hasRole` vs missing `ROLE_` prefix, missing **CSRF token**, broad rule first, wrong chain matched

Mistake	Symptom → fix
catch-all chain ordered first	specific chain ignored → `@Order`, specific first
`hasRole("ROLE_X")`	startup `IllegalArgumentException` → `hasRole("X")`
`csrf.disable()` with session login	CSRF hole → send token / `csrf.spa()`
`anyRequest().permitAll()` last	new endpoints public → `denyAll()` / `authenticated()`
own filter as `@Component`	runs twice → `addFilterBefore` + disabled `FilterRegistrationBean`
context lost in `@Async`	`AuthenticationCredentialsNotFoundException` → `DelegatingSecurityContextAsyncTaskExecutor`
Actuator under app rules	`/actuator/env` public → own chain/rules
exception forwarded to `/error` not permitted	real 500 shows as 401/403 → permit `/error`

Code to remember

```
@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth                        // first match wins
            .requestMatchers("/", "/public/**", "/login").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")         // authority ROLE_ADMIN
            .anyRequest().authenticated())
        .formLogin(form -> form.loginPage("/login").permitAll())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

In the interview

Q: Walk me through a request in a Spring Security app. A: `DelegatingFilterProxy` → `FilterChainProxy` → first matching chain; context loaded, headers, CSRF, authentication filters, anonymous token, `ExceptionTranslationFilter`, then `AuthorizationFilter` decides.

Q: Where is the user stored during and between requests? A: During: `SecurityContextHolder` (ThreadLocal). Between: the HTTP session (form login), or nowhere on stateless chains.

Q: 401 vs 403? A: 401 = unauthenticated (entry point), 403 = authenticated but denied (access-denied handler); a missing CSRF token also gives 403.

Q: `hasRole("ADMIN")` vs `hasAuthority("ADMIN")`? A: The first checks `ROLE_ADMIN`, the second the literal `ADMIN`.

Q: When can you disable CSRF? A: Only on chains that never authenticate with browser-automatic credentials, e.g. pure bearer-token APIs.

Q: "Users can only see their own orders"? A: Method security with an ownership expression (`@PostAuthorize("returnObject.owner == authentication.name")`) or scope the query by the current user.

Q: How would you add API-key login? A: A filter/`AuthenticationConverter` building an `ApiKeyAuthenticationToken` + an `AuthenticationProvider`, registered with `addFilterBefore(..., BasicAuthenticationFilter.class)`.

Gotchas

- Plain/SHA-256/MD5 password hashes → cracked on leak; use adaptive salted hashes.
- `roles()` then `authorities()` on `User` → roles vanish; one call with the full list.
- CORS configured only in MVC → preflight 401.
- Method security on a self-invoked or private method → not protected.
- Debug with `logging.level.org.springframework.security=TRACE` (every filter + the decision).

TL;DR

Filter chain → authentication → `SecurityContextHolder` → `AuthorizationFilter`. Lambda DSL only in Security 7; first matching rule wins; roles are `ROLE_` authorities; 401 = who are you, 403 = no; CSRF stays on wherever cookies authenticate.

11. Spring Security 2: Tokens and OAuth2

JWT and bearer tokens, OAuth 2.0/OIDC, a JWT resource server, claims to authorities, opaque tokens, multi-tenancy, OAuth2 login, service-to-service calls, refresh and revocation, the BFF pattern, Authorization Server, rate limiting.

Must-know

- **JWT** — `header.payload.signature`; claims are readable (base64), the signature proves origin and integrity. Prefer asymmetric keys (RS256/ES256) published as a **JWKS**; short lifetimes; verify `exp`, `iss`, `aud`.
- **OAuth 2.0** — delegated authorization: a client gets an **access token** to call an API. **OIDC** — identity layer on top: **ID token** (JWT about the user) + UserInfo; answers "who logged in".
- **Grants** — authorization code + **PKCE** for users; **client credentials** for services; refresh tokens for renewal; implicit and password grants obsolete (Security 7 removed password-grant support).
- **Resource server** — `oauth2ResourceServer(rs -> rs.jwt(withDefaults()))` + `spring.security.oauth2.resourceserver.jwt.issuer-uri`; `BearerTokenAuthenticationFilter` → `JwtDecoder` (Nimbus, JWKS via discovery, key by `kid`) → validators `exp`/`nbf` (**60 s skew**), `iss`, `typ`, your `aud` → `JwtAuthenticationConverter`.
- **Authorities** — default `scope` claim → `SCOPE_x`; roles/groups need a converter or `jwt.authorities-claim-name` + `jwt.authority-prefix` (these replace the scope mapping).

- **Opaque tokens** — random reference checked via introspection (RFC 7662): instant revocation + privacy, cost = network call (cache briefly).
- **Multi-tenancy** — `JwtIssuerAuthenticationManagerResolver` with an **allow-list** of issuers.
- **OAuth2 login** — `oauth2Login()`: code flow with `state` (CSRF for the callback), `nonce`, PKCE; tokens stay server-side, browser gets a session.
- **Service-to-service** — `OAuth2AuthorizedClientManager` + `OAuth2ClientHttpRequestInterceptor` on `RestClient` (or `@ClientRegistrationId` on HTTP interfaces); relay or exchange the user's token when user context matters; mTLS in a mesh.
- **BFF** — tokens stay server-side; SPA gets an `HttpOnly`, `Secure`, `SameSite` session cookie + CSRF protection. Tokens in `localStorage` = any XSS steals them.
- **Logout with stateless tokens** — short access tokens, revoke the refresh token, end the local session, OIDC RP-initiated logout.
- **`NimbusJwtEncoder.withKeyPair(...)` / `withSecretKey(...)`** — mint tokens for tests/demos; real users → an authorization server (Spring Authorization Server now inside Spring Security 7).

Key facts & numbers

	Sessions	Tokens (JWT)
State	server stores login; browser holds `JSESSIONID`	client sends signed token every call
Logout	instant	hard — short lifetimes
Needs	CSRF protection; shared session store when scaled (Spring Session)	key distribution (JWKS)
Best for	browser apps / BFF	APIs, service-to-service, mobile

	JWT	Opaque
Validation	local, fast	introspection call per request (cache)
Revocation	only at expiry	instant
Claims	exposed to the holder	hidden

Result	Meaning
401 `invalid_token`	bad/missing/expired token, wrong issuer/audience
403 `insufficient_scope`	valid token, missing authority

Mistake	Fix
no audience check → tokens for other APIs accepted	`jwt.audiences` or `JwtClaimValidator("aud", …)`
custom validator **replacing** defaults → expired tokens pass	combine with `JwtValidators.createDefaultWithIssuer(...)`
`hasAuthority("messages.read")` → always 403	`SCOPE_messages.read`
`jwt()` tests without your converter	pass `.authorities(yourConverter)`
accepting any `iss` (multi-tenant)	issuer allow-list
ID token sent to APIs	send the **access** token
private key in properties/Git	vault/KMS; rotate via `kid`

Code to remember

```
@Bean @Order(1)
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")
        .authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.GET, "/api/**").hasAuthority("SCOPE_orders.read")
            .anyRequest().authenticated())
        .oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()))
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(csrf -> csrf.disable());            // bearer headers, no cookies
    return http.build();
}
```

In the interview

Q: How does a resource server validate a JWT? A: Filter extracts the bearer token; `JwtDecoder` fetches the issuer's JWKS, picks the key by `kid`, verifies the signature; validators check `exp`/`nbf` (60 s), `iss`, `typ`, `aud`; scopes become `SCOPE_*` authorities.

Q: OAuth 2.0 vs OIDC? A: Delegated authorization with access tokens vs identity layer adding the ID token and UserInfo; "Sign in with Google" is OIDC code flow + PKCE.

Q: JWT or opaque? A: JWT for fast local validation; opaque for instant revocation or hidden claims, at the cost of introspection.

Q: Why PKCE if the client has a secret? A: It binds the code to the party that started the flow, so a stolen code is useless; OAuth 2.1 requires it and Security 7 sends it by default.

Q: Where should a browser SPA keep tokens? A: Nowhere — a BFF keeps them server-side and gives the SPA an `HttpOnly` cookie with CSRF protection.

Q: How does one service call another securely? A: Client credentials via `OAuth2AuthorizedClientManager` + interceptor on `RestClient`; relay/exchange the user token when identity matters.

Q: Is the security context available in `@Async`? A: Not by default (ThreadLocal); wrap the executor in `DelegatingSecurityContextAsyncTaskExecutor`; in WebFlux it lives in the Reactor `Context`.

Gotchas

- Long-lived JWT access tokens → stolen token valid for days; minutes + refresh tokens.
- CSRF disabled on a BFF / OAuth2-login app → it uses a cookie session; keep CSRF (`csrf.spa()`).
- Rolling your own token server for real users → use an authorization server.

Must-know — security fundamentals (§11.18)

- **Encoding / hashing / encryption / tokenisation** — reversible without a key (Base64) / one-way fingerprint (SHA-256; passwords → bcrypt/Argon2) / reversible with a key (AES-GCM) / swap for a vault reference (card tokens).
- **Symmetric vs asymmetric** — one fast shared key (AES) vs public/private pair (RSA, X25519, Ed25519) for key exchange and signatures; real systems are **hybrid**.
- **Signature** — private key signs a hash, public key verifies origin + integrity (RS256/ES256 JWTs, certificates, SSH, passkeys).
- **TLS 1.3** — ClientHello/ServerHello exchange ephemeral ECDHE key shares (1 RTT) → session keys; server signs the handshake with its certificate key; client checks chain + hostname; AES-GCM for data; **forward secrecy**. mTLS adds a client certificate.
- **SSH keys** — public key in `authorized_keys`; login = sign the server's challenge; `known_hosts` pins the server's host key.
- **TOTP** — `HMAC-SHA1(secret, floor(t/30))` → dynamic truncation → 6 digits; accept ±1 step; phishable.
- **Passkeys (WebAuthn)** — per-site key pair; server stores only the public key; signature bound to the origin → phishing-resistant. Spring: `http.webAuthn(...)`, `FACTOR_WEBAUTHN`.
- **SSO** — SAML 2.0 (signed XML assertion via browser POST, `saml2Login()`) vs OIDC (signed JWT ID token over OAuth 2.0 code + PKCE, `oauth2Login()`).

Q: How does HTTPS set up keys without sending them? **A:** Ephemeral (EC)DH: each side sends a public key share and combines it with its own private share to get the same secret; the certificate signature proves who the server is.

Q: Hashing vs encryption? **A:** Hashing is one-way and keyless (integrity, password storage); encryption is reversible with a key (confidentiality).

Q: Why are passkeys better than passwords + TOTP? **A:** No shared secret on the server and the signed challenge is bound to the real origin, so phishing sites and database leaks get nothing usable.

TL;DR

Resource server = `issuer-uri` + signature/`exp`/`iss`/`aud` checks, `scope` → `SCOPE_x`; 401 invalid token vs 403 insufficient scope. Code + PKCE for users, client credentials for services, BFF for browsers, short tokens + refresh.

12. Testing Spring Applications

The test pyramid, Boot 4 test starters, unit tests, `@SpringBootTest`, slices, `MockMvcTester`, `RestTestClient`, `@MockitoBean`, context caching (measured), `@DynamicPropertySource`, Testcontainers and `@ServiceConnection`.

Must-know

- **Pyramid** — many Spring-free unit tests (ms) → slices for layer behaviour (~0.1 s context) → a few `@SpringBootTest`s (~0.7 s context) → very few real-port tests.
- **Boot 4 test starters** — one per technology (`spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-restclient-test` …); slice annotations moved to per-module packages; JUnit 6, AssertJ, Mockito, Awaitility inside `spring-boot-starter-test`.
- **`@SpringBootTest`** — whole context; `webEnvironment = MOCK` (default), `RANDOM_PORT`, `DEFINED_PORT`, `NONE`. In Boot 4 **no longer** gives MockMvc/`RestTestClient`/`WebTestClient`/`TestRestTemplate` without `@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` / `@AutoConfigureWebTestClient` / `@AutoConfigureTestRestTemplate`.
- **Slice** — starts only one layer's auto-configurations + matching beans; mock the rest.
- **`@MockitoBean` / `@MockitoSpyBean`** (Framework 6.2) — mock/spy **in the context**; replaced Boot's `@MockBean`/`@SpyBean` (deprecated 3.4, **removed in 4.0**); reset after each test; part of the cache key; not allowed inside `@Configuration` classes. `@Mock` = plain Mockito, no Spring.
- **`MockMvcTester`** (Framework 6.2) — AssertJ-style MockMvc: `mvc.get().uri(..).exchange()`, `assertThat(result).hasStatusOk().bodyJson()`.
- **`RestTestClient`** (Framework 7) — one fluent client for MockMvc or a running server; Boot 4 injects it only with `@AutoConfigureRestTestClient` (even in `@WebMvcTest`). `WebTestClient` for WebFlux.
- **Context cache** — key = merged configuration (classes, profiles, properties, `@MockitoBean`s, `@DynamicPropertySource`, `@AutoConfigure…`, `webEnvironment`); **max 32**, LRU; `@DirtiesContext` evicts; Framework 7 **pauses** inactive contexts (stops `SmartLifecycle` beans like schedulers/listeners).
- **Test transactions** — `@Transactional` tests roll back after each method (`@Commit` to keep); `@DataJpaTest` is transactional; with `RANDOM_PORT` the request runs on a server thread in another transaction → **not** rolled back.
- **`@DynamicPropertySource`** — lazy suppliers for runtime values, highest precedence. **Testcontainers + `@ServiceConnection`** (Boot 3.1) — connection details from the container, no URL properties.
- **`ApplicationContextRunner`** — tiny context in ms for auto-configuration/conditions (`hasSingleBean`, `hasFailed`).
- **Security tests** — `@WithMockUser(roles = "ADMIN")`, `user()`, `httpBasic()`, `jwt().authorities(...)`, `csrf()`; test the 401/403 paths too.

Key facts & numbers

Test slices:

Slice	Test starter	Loads	Rolled back?	Extras
`@WebMvcTest(Controller.class)`	`-webmvc-test`	MVC, Jackson, validation, security filters, controllers/advice	no	`MockMvcTester`, `MockMvc`
`@WebFluxTest`	`-webflux-test`	WebFlux, codecs, controllers	no	`WebTestClient`
`@DataJpaTest`	`-data-jpa-test`	JPA, repositories, embedded DB, Flyway	**yes**	`TestEntityManager`
`@JdbcTest` / `@DataJdbcTest`	`-jdbc-test` / `-data-jdbc-test`	`DataSource`, `JdbcClient` / Data JDBC	**yes**	—
`@DataR2dbcTest`	`-data-r2dbc-test`	R2DBC repositories	**no**	—
`@JsonTest`	`-jackson-test`	`JsonMapper` + customisations	no	`JacksonTester`
`@RestClientTest(Client.class)`	`-restclient-test`	`RestClient.Builder`, the client	no	`MockRestServiceServer`
`@DataMongoTest` / `@DataRedisTest`	`-data-mongodb-test` / `-data-redis-test`	store repositories	no	—

`@AutoConfigureTestDatabase(replace = NON_TEST)` (default since 3.4) replaces your DB unless it is already a test DB (e.g. a `@ServiceConnection` container); `NONE` keeps it. `@WebMvcTest` in the book: 127 beans vs 253 for the full context.

Client	Stack	Needs a server?
`MockMvcTester`	MVC	no
`RestTestClient`	MVC (or any via `bindToServer`)	optional
`WebTestClient`	WebFlux (also MockMvc)	optional
`TestRestTemplate`	any	yes

Measured context cache (7 classes): 4 builds; each miss ~0.5 s, each hit **0 ms**; new `@MockitoBean`, a different property and `@DirtiesContext` each caused a miss.

Code to remember

```
@WebMvcTest(BookController.class)
class BookControllerTests {
    @Autowired MockMvcTester mvc;
    @MockitoBean BookService books;

    @Test void returns404AsProblemDetail() {
        given(books.find(9)).willThrow(new BookNotFoundException(9));
        assertThat(mvc.get().uri("/api/books/9")).hasStatus(HttpStatus.NOT_FOUND)
            .bodyJson().extractingPath("$.bookId").isEqualTo(9);
    }
}
```

In the interview

Q: How do you test a Boot REST API? A: Unit tests for services (Mockito, no Spring); `@WebMvcTest` + `MockMvcTester` + `@MockitoBean` per controller; a few `@SpringBootTest`s (`@AutoConfigureMockMvc`, one `RANDOM_PORT` with `RestTestClient`); `@DataJpaTest` against PostgreSQL via Testcontainers.

Q: Does `@WebMvcTest` load `@Service` beans? A: No — mock them with `@MockitoBean` or `@Import` them.

Q: Why is our test suite slow? A: Too many contexts: every distinct set of mocks/properties/profiles and every `@DirtiesContext` builds one; log `org.springframework.test.context.cache` at DEBUG, standardise a base class, use slices.

Q: Why might a `@Transactional` test give false confidence? A: It rolls back, so commit-time behaviour (deferred constraints, AFTER_COMMIT listeners, lazy loading outside the session) never runs, and in `RANDOM_PORT` tests it doesn't cover the server's transaction.

Q: How do you test against a real database? A: Testcontainers + `@ServiceConnection` in one shared configuration so the cache reuses the container; pin the production image version.

Q: What's new for testing in Boot 4 / Framework 7? A: Per-tech test starters, moved packages, `@MockBean` removed, explicit `@AutoConfigure…` for test clients, `RestTestClient`, context pausing, JUnit 6 (JUnit 4 support deprecated).

Gotchas

- "Unable to find a @SpringBootConfiguration" → test package not under the main class's package.
- Slice loads services/repositories → custom `@ComponentScan` on the main class; move it to a separate `@Configuration`.
- Test passes, prod throws `LazyInitializationException` → test transaction kept the session open.
- CI-only failures → fixed ports, shared DBs, time/order dependence; use `RANDOM_PORT`, a fixed `Clock` bean.
- `@DataR2dbcTest` does not roll back like `@DataJpaTest`.
- Waiting with `Thread.sleep` → Awaitility `await().atMost(...)`.

Must-know — types of API testing (§12.18)

Type	Question it answers	Spring / Java tool
Smoke	Does it start and answer at all?	one `@SpringBootTest` context load + `/actuator/health` after deploy
Functional	Does each endpoint do what the spec says?	`@WebMvcTest` + `MockMvcTester`, `@MockitoBean`
Integration	Do the real parts work together?	`@SpringBootTest`, `@DataJpaTest`, Testcontainers `@ServiceConnection`
Contract	Do provider and consumers agree on the API?	Pact, OpenAPI schema validation
Regression	Did a change break what worked?	the whole automated suite in CI, one test per fixed bug
Load / stress / soak	Does it meet latency at expected load; where does it break; does it leak over hours?	Gatling, k6, JMeter + Micrometer metrics
Security / fuzz	Does it refuse bad users and bad input?	`spring-security-test` (`jwt()`, `@WithMockUser`), OWASP ZAP, jqwik/Jazzer, Schemathesis
UI / end-to-end	Does a real user journey work through the UI?	Playwright/Selenium against `RANDOM_PORT` or a staging URL

Q: Load vs stress vs soak test? **A:** Load = expected peak, check SLOs; stress = push past it to find the breaking point and failure mode; soak = normal load for hours to find leaks and slow degradation.

Q: Where do contract tests fit? **A:** Between services: consumers publish expectations, the provider verifies them in its build, so both sides can release independently without full end-to-end environments.

TL;DR

Unit → slices → few `@SpringBootTest`s. Boot 4: per-technology test starters, `@MockitoBean` (not `@MockBean`), explicit `@AutoConfigure…` for test clients. Keep context configurations few so the 32-entry cache hits.

13. Reactive Programming and Spring WebFlux

Reactive Streams, `Mono`/`Flux` and operators, `StepVerifier`, errors, backpressure, schedulers, WebFlux controllers and functional routes, SSE, `WebClient`, R2DBC, virtual threads vs WebFlux (measured).

Must-know

- **Why reactive** — thread-per-request wastes threads on waiting; event loops (about one thread per core on Netty) never wait.
- **Reactive Streams** — `Publisher`, `Subscriber`, `Subscription`, `Processor` (JDK copy: `java.util.concurrent.Flow`); **backpressure** = subscriber's `request(n)`; exactly one terminal signal (complete or error).
- `Mono` 0..1, `Flux` 0..N; **lazy and cold — nothing happens until subscribe** (WebFlux subscribes for you). `just(expr)` evaluates eagerly; `defer`/`fromCallable` are lazy (retries repeat them).
- `map` **vs** `flatMap` **vs** `concatMap` — sync function vs async (returns a publisher), concurrent and **interleaved** (default concurrency 256) vs one-at-a-time, ordered; `flatMapSequential` = concurrent, source order.
- **Errors end the stream** — `onErrorReturn`/`onErrorResume`/`onErrorMap`, `retryWhen(Retry.backoff(3, 1s))`, `timeout` (before `retryWhen` = per attempt).
- `publishOn` **vs** `subscribeOn` — `publishOn` switches the thread for operators **after** it; `subscribeOn` chooses where the **source** runs, wherever it appears.
- **Never block an event loop** — JDBC, `Thread.sleep`, `block()` stall every request on it; Reactor throws `IllegalStateException` for `block()` on non-blocking threads. Unavoidable blocking: `Mono.fromCallable(...).subscribeOn(Schedulers.boundedElastic())`; better: R2DBC, `WebClient`.
- **Context, not ThreadLocal** — security (`ReactiveSecurityContextHolder`), transactions (`ReactiveTransactionManager`/`R2dbcTransactionManager`, `TransactionalOperator`) and MDC live in the Reactor `Context`.
- **R2DBC** — non-blocking SQL, not an ORM; `ReactiveCrudRepository`, `DatabaseClient`; `@DataR2dbcTest` does not roll back. JPA/JDBC are blocking.
- **WebFlux** — annotated controllers returning `Mono`/`Flux` or functional `RouterFunction`s; SSE with `text/event-stream`; tests `@WebFluxTest` + `WebTestClient`; Boot 4 needs `spring-boot-starter-webclient` for `WebClient.Builder`.
- **Virtual threads** (`spring.threads.virtual.enabled=true`, Java 21+, Boot 3.2+) — make **waiting** cheap, not CPU work faster; **pinning** = can't unmount (before Java 24 inside `synchronized`; still native frames).

Key facts & numbers

MVC + virtual threads vs WebFlux (1,000 concurrent clients, 100 ms I/O endpoint, Apple M4 Pro):

Mode	req/s	p50	Peak platform threads
MVC, 200 platform threads	1,909	517 ms	231
MVC + virtual threads	**7,942**	108 ms	33
WebFlux non-blocking	**8,058**	110 ms	56
WebFlux + one `Thread.sleep` (bad; 100 clients)	**97**	724 ms	56

Situation	Choose
typical CRUD/REST on JDBC/JPA, blocking clients	**MVC + virtual threads** (default)
streaming: SSE, WebSockets, NDJSON, end-to-end backpressure	**WebFlux**
gateways/proxies forwarding bytes	WebFlux
heavy fan-out composition	either (Reactor operators are expressive)
CPU-bound work	neither helps
team new to reactive, blocking deps	MVC + virtual threads

Either way the **DB pool** is still the real limit.

Reactor operators:

Goal	Operator	Remember
create lazily	`fromCallable`, `defer`, `fromFuture`	per subscription
async per item	`flatMap(fn[, concurrency])`	concurrent, interleaved
async, ordered	`concatMap` / `flatMapSequential`	sequential / buffers
combine in parallel	`Mono.zip(a, b, c)`	like `Promise.all`
empty fallback	`switchIfEmpty`, `defaultIfEmpty`	the 404 path
aggregate	`collectList`, `reduce`, `count`	returns `Mono`
then	`then()`	do A, then B
side effects	`doOnNext`, `doOnError`, `doFinally`, `log()`	not business logic
retry	`retryWhen(Retry.backoff(3, 1s))`	attempts at 0/1/3/7 s (no jitter)
backpressure	`limitRate(n)` (re-requests at 75 %), `onBackpressureDrop/Latest/Buffer(n)`	unbounded buffers → OOM
threads	`publishOn` / `subscribeOn`; `boundedElastic()` for blocking	
hot / shared	`share()`, `cache()`, `Sinks.many().multicast()`	late subscribers miss items
context	`contextWrite`, `deferContextual`	reactive ThreadLocal
test	`StepVerifier.create(p).expectNext(..).verifyComplete()`, `withVirtualTime` + `thenAwait`	build publisher inside the supplier

Code to remember

```
Mono<OrderView> view(long id) {
    return Mono.zip(orders.find(id), customers.findByOrder(id), prices.forOrder(id))   // parallel
               .map(t -> new OrderView(t.getT1(), t.getT2(), t.getT3()))
               .timeout(Duration.ofSeconds(2))
               .switchIfEmpty(Mono.error(new OrderNotFound(id)));
}
StepVerifier.create(view(1)).expectNextCount(1).verifyComplete();
```

In the interview

Q: WebFlux or MVC with virtual threads? A: MVC + virtual threads for I/O-bound CRUD — same ~8,000 req/s as WebFlux in the book's test, simpler code and debugging; WebFlux for streaming, gateways, end-to-end backpressure.

Q: What happens if you block inside a WebFlux handler? A: You stall an event-loop thread and every request on it — measured 97 req/s; move blocking work to `boundedElastic` or use non-blocking drivers.

Q: `publishOn` vs `subscribeOn`? A: `publishOn` moves downstream operators; `subscribeOn` moves the source regardless of position.

Q: What is backpressure? A: The subscriber's `request(n)` limits what the publisher sends; `limitRate`, `onBackpressureDrop/Latest/Buffer` for sources that can't slow down.

Q: Can you use JPA with WebFlux? A: Not without blocking; use R2DBC, or MVC with virtual threads.

Q: How do transactions and security work reactively? A: Carried in the Reactor `Context` — `ReactiveTransactionManager` / `TransactionalOperator`, `ReactiveSecurityContextHolder` — not ThreadLocals.

Q: Do virtual threads make code faster? A: No — they make waiting cheap so more requests can wait at once; CPU-bound work gains nothing.

Gotchas

- Pipeline built but not returned/subscribed → nothing happens.
- `map` with an async function → `Flux<Mono<T>>`, inner calls never run → `flatMap`.
- `Mono.just(expensiveCall())` → runs eagerly, retry doesn't repeat it → `fromCallable` / `defer`.
- `ThreadLocal`-based MDC/security/tx → lost after a thread hop.
- Unbounded `flatMap` / `onBackpressureBuffer()` → overwhelmed downstream, memory growth.
- Both MVC and WebFlux starters → Boot silently starts MVC.
- Virtual-time test without `thenAwait` → hangs.

TL;DR

`Mono` / `Flux` are lazy; `flatMap` is concurrent, `concatMap` ordered; never block an event loop; context not ThreadLocal. Measured: MVC + virtual threads 7,942 vs WebFlux 8,058 req/s vs 1,909 on platform threads — default to MVC + virtual threads, WebFlux for streaming.

14. Messaging, Async and Scheduling

Events and `@TransactionalEventListener`, `@Async` and executors (virtual threads), `@Scheduled`, Kafka producers/consumers, `@EmbeddedKafka`, dead-letter topics, `@RetryableTopic`, idempotence and transactions, the outbox, RabbitMQ.

Must-know

- **Events** — `@EventListener` synchronous; `@Async @EventListener` gets no transaction and no exception propagation; `@TransactionalEventListener` phases `BEFORE_COMMIT` (can veto the commit) / `AFTER_COMMIT` (default) / `AFTER_ROLLBACK` / `AFTER_COMPLETION`. Usual combo: `@Async` + `AFTER_COMMIT` + `REQUIRES_NEW` — still not durable.
- **`@Async`** — `@EnableAsync` + proxy + executor; returns `void` or `CompletableFuture<T>`. Executor: `@Async("name")` → `AsyncConfigurer` → Boot's `applicationTaskExecutor` (`ThreadPoolTaskExecutor`, **8 core threads, unbounded queue**, `spring.task.execution.*`) or a virtual-thread executor. Declaring your own `Executor` bean removes Boot's → `@Bean(defaultCandidate = false)`.
- **Async exceptions** — future return type: stored in the future; `void`: `AsyncUncaughtExceptionHandler` (logs only). **Context** — MDC via `TaskDecorator` / `spring.task.execution.propagate-context=true`; security via `DelegatingSecurityContextAsyncTaskExecutor`; transactions never follow.
- **Virtual threads** — `spring.threads.virtual.enabled=true` → one virtual thread per task, no pool, no queue; limit concurrency yourself (`@ConcurrencyLimit`).
- **`@Scheduled`** — `@EnableScheduling`; `fixedRate` start-to-start, `fixedDelay` end-to-end (use for polling an outbox), `cron` **six fields** starting with seconds (+ `zone`). Boot's scheduler has **one thread** (`spring.task.scheduling.pool.size`). Runs on **every instance** → ShedLock, leader election, Kubernetes CronJob; idempotent anyway.
- **Kafka basics** — key → partition → **order only within a partition** (key by aggregate id); a consumer group splits partitions among members; every group gets every record. Boot 4: `spring-boot-starter-kafka`, `KafkaTemplate.send(topic, key, value)` → `CompletableFuture<SendResult>`, `@KafkaListener(topics, groupId)`, `JacksonJsonSerializer` (Jackson 3).

- **Delivery** — at-least-once: spring-kafka disables auto-commit and commits after the listener succeeds (ack mode `BATCH`) → make consumers **idempotent** (processed-id table, upserts). Producer idempotence (on by default) only stops duplicates from producer retries.
- **Exactly-once** — Kafka transactions + `read_committed` for Kafka-to-Kafka only; anything with a DB → outbox + idempotent consumers.
- **Transactional outbox** — write the event row in the same DB transaction; relay by a `fixedDelay` poller (`SELECT … FOR UPDATE SKIP LOCKED`) or CDC (Debezium); at-least-once, consumers dedupe. Kafka transactions can't cover the DB commit (dual-write problem).
- **RabbitMQ** — exchanges → bindings → queues, per-message acks, prefetch, DLX (`default-requeue-rejected=false`), publisher confirms; Boot 4 retry property `retry.max-retries` (was `max-attempts`, which counted the first call).

Key facts & numbers

Kafka error handling — DLT and retries:

Piece	Behaviour
`DefaultErrorHandler` default	`FixedBackOff(0, 9)` = 10 attempts, no delay → set a real back-off
`DeadLetterPublishingRecoverer`	after the last attempt → **`<topic>-dlt`** (since spring-kafka 3.3; old `<topic>.DLT`), same partition number, cause in headers
non-retryable exceptions	e.g. `IllegalArgumentException` marked not retryable → 1 attempt → DLT
poison pill	`ErrorHandlingDeserializer` → fatal `DeserializationException` → listener never called, bytes to DLT
blocking back-off limit	total must stay well under `max.poll.interval.ms`

	`DefaultErrorHandler` (blocking)	`@RetryableTopic` (non-blocking)
Partition during retries	stopped	keeps flowing
Ordering	kept	lost for the retried record
Extra topics	DLT only	retry topic(s) + DLT
Long delays (minutes)	dangerous	fine
Choose when	order matters (balances)	independent records, slow-recovering deps

	Kafka	RabbitMQ
Model	partitioned, replicated **log**; offsets	**queue broker**; acked messages deleted
Replay / many readers	yes (group offsets)	no (queue per reader)
Routing	topic + partition	direct, topic, fanout, headers
TTL, priority, delay	no	yes
Test without Docker	`@EmbeddedKafka` (in-JVM KRaft)	no (Testcontainers)
Choose for	event streaming, high throughput	task distribution, complex routing

Versions: Boot 4.1.1 → spring-kafka 4.1.1, kafka-clients 4.2.1, Spring AMQP 4.1.1.

Code to remember

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);        // → orders-dlt
    var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 2)); // 1 + 2 retries
    handler.addNotRetryableExceptions(IllegalArgumentException.class);
    return handler;
}
@KafkaListener(topics = "orders", groupId = "billing")
void on(OrderPlaced e) { if (processed.add(e.id())) billing.charge(e); }   // idempotent
```

In the interview

Q: How does `@Async` work and what are its pitfalls? A: Proxy submits to an executor and returns at once; pitfalls: self-invocation, `void` exceptions only reach the handler, unbounded default queue, no transaction/security context.

Q: `fixedRate` vs `fixedDelay` vs `cron`? A: Start-to-start vs end-to-start vs six-field calendar; one scheduler thread by default so slow jobs delay others.

Q: A job runs on every instance — how do you run it once? A: ShedLock (DB table) or a Redis lock, leader election, or an external scheduler; idempotent job anyway.

Q: A message keeps failing? A: `DefaultErrorHandler` with limited back-off + `DeadLetterPublishingRecoverer` → `<topic>-dlt`; non-retryable exceptions straight to DLT; `@RetryableTopic` when order doesn't matter.

Q: How do you update the DB and publish reliably? A: Transactional outbox + relay (poller or CDC) + idempotent consumers.

Q: Is Kafka exactly-once? A: Idempotent producer + transactions + `read_committed` for Kafka-to-Kafka; with a database you need outbox + dedupe.

Q: How do you keep one order's events in sequence? A: Key by order id (one partition) and use blocking retries, not retry topics.

Q: How do you test Kafka without Docker? A: `@EmbeddedKafka`, `spring.kafka.bootstrap-servers=${spring.embedded.kafka.brokers}`, Awaitility.

Gotchas

- No `@EnableAsync`/`@EnableScheduling` → `@Async` runs synchronously, `@Scheduled` never runs.
- Sending to Kafka inside a DB transaction → ghost or lost events → outbox.
- No `ErrorHandlingDeserializer` → a poison pill blocks the partition.
- Looking for `orders.DLT` on spring-kafka 3.3+ → it's `orders-dlt`.
- Non-idempotent consumers → double charges after rebalance/redelivery.
- On virtual threads a slow fixed-rate job can overlap itself (not on platform threads).
- `Thread.sleep` in async tests → Awaitility with a timeout.

TL;DR

`@Async`/`@Scheduled` are proxy/executor features needing `@Enable…`; one scheduler thread; one run per instance. Kafka: key → partition → order; at-least-once → idempotent consumers; `DefaultErrorHandler` + DLT (`<topic>-dlt`) or `@RetryableTopic`; outbox for DB + event.

15. Observability and Production

Actuator, health groups and probes, Micrometer metrics, the Observation API, tracing, structured logging, graceful shutdown, virtual threads, extracted jars / AOT cache / native images, a production checklist.

Must-know

- **Observability** = logs + metrics + traces, linked by the **trace id**. Spring: Logback, **Micrometer**, **Micrometer Tracing** + OpenTelemetry (or Brave) bridge, all fed by the **Observation API**.
- **Actuator** — endpoints under `/actuator`; over HTTP **only** `health` is exposed by default; `exposure.include` + per-endpoint `access` (`none`/`read-only`/`unrestricted`); secure with `EndpointRequest.to(...)`/`toAnyEndpoint()` (Boot 4 package `org.springframework.boot.security.autoconfigure.actuate.web.servlet`), ideally on a separate `management.server.port`.
- **Liveness vs readiness** — "restart me?" vs "send me traffic?"; `/actuator/health/liveness` and `/readiness` **on by default in Boot 4**; readiness only after startup, refusing traffic during shutdown; **never** external deps (DB) in liveness. Out of rotation: publish `AvailabilityChangeEvent` with `ReadinessState.REFUSING_TRAFFIC`. `DOWN`/`OUT_OF_SERVICE` → HTTP 503.
- **Custom health** — a `HealthIndicator` bean (`Health.up()/down().withDetail(...)`), Boot 4 package `org.springframework.boot.health.contributor`.
- **Micrometer** — vendor-neutral facade ("SLF4J for metrics"): `Counter` (only increases), `Gauge` (current value), `Timer` (count + total + max + percentiles/histograms), `DistributionSummary`, `LongTaskTimer`. **Low-cardinality tags** — never user id or raw URL.
- **Observation API / `@Observed`** — instrument once → timer + span + log correlation; `@Observed` needs `management.observations.annotations.enabled=true` + `spring-boot-starter-aspectj`.

- **Tracing** — spans for incoming/outgoing calls; W3C `traceparent` propagation (and Kafka headers) via Boot's injected client builders; `[traceId-spanId]` in logs; sampling **`0.1`**; export over OTLP; Boot 4 `spring-boot-starter-opentelemetry` (Brave/Zipkin: `spring-boot-starter-zipkin`). Successor of Spring Cloud Sleuth since Boot 3.
- **Structured logging** (Boot 3.4+) — `logging.structured.format.console=ecs|logstash|gelf`, JSON per event with MDC and trace ids.
- **Graceful shutdown** (default since 3.4) — on SIGTERM stop accepting, drain in-flight within `spring.lifecycle.timeout-per-shutdown-phase` (**`30 s`**), close context; Kubernetes: `preStop` sleep, grace period > drain timeout.
- **Startup** — extracted jar + JDK **AOT cache** (JEP 483, Java 24; JEP 514 simplifies in 25; training run with `-Dspring.context.exit=onRefresh`) — **`934 → 467 ms`** in the book; or Spring AOT + **GraalVM native image** (tens of ms, less memory; long builds, closed world, conditions fixed at build time, lower peak throughput possible).
- **Images** — `mvn spring-boot:build-image` (buildpacks) or layered Dockerfile (`java -Djarmode=tools -jar app.jar extract --layers`); non-root; `-XX:MaxRAMPercentage`.

Key facts & numbers

Actuator & observability properties:

Property	Default	Meaning
`management.endpoints.web.exposure.include`	`health`	endpoints reachable over HTTP
`management.endpoint.<id>.access` / `endpoints.access.default`	per endpoint (`shutdown` = `none`)	`none`, `read-only`, `unrestricted`
`management.server.port`	same port	separate management port
`management.endpoint.health.show-details`	`never`	`when-authorized`, `always`
`management.endpoint.health.probes.enabled`	`true` (Boot 4)	liveness/readiness groups
`management.endpoint.env.show-values`	`never`	mask `env` / `configprops`
`management.metrics.tags.<key>`	—	common tag on every meter
`management.metrics.distribution.percentiles-histogram.<meter>`	—	buckets for p95/p99
`management.observations.annotations.enabled`	`false`	enable `@Observed`, `@Timed`, `@Counted`
`management.tracing.sampling.probability`	`0.1`	share of traces recorded
`management.tracing.export.enabled`	`true`	was `management.tracing.enabled`
`management.opentelemetry.tracing.export.otlp.endpoint`	—	was `management.otlp.tracing.endpoint`
`management.tracing.propagation.type`	W3C	`b3` for legacy Zipkin
`logging.structured.format.console`	—	`ecs`, `logstash`, `gelf`
`spring.lifecycle.timeout-per-shutdown-phase`	`30s`	graceful drain time
`spring.threads.virtual.enabled`	`false`	virtual threads for Tomcat, `@Async`, `@Scheduled`, listeners

Endpoint	Shows / does
`health`	UP/DOWN/OUT_OF_SERVICE/UNKNOWN; groups liveness, readiness
`info`, `metrics`, `prometheus`	build/git info; meters JSON; Prometheus text (needs `micrometer-registry-prometheus`)
`loggers`	change levels at runtime: `POST {"configuredLevel":"DEBUG"}`
`env`, `configprops`	property sources, bound properties (masked)
`conditions`, `beans`, `mappings`	auto-config report, beans, request mappings
`threaddump`, `heapdump`	heap dumps can contain secrets
`startup`	startup steps (needs `BufferingApplicationStartup`)
`shutdown`	access `none` by default

Production checklist: expose only `health,info,prometheus` · probes wired, no deps in liveness · Prometheus/OTLP registry, common tags, histograms · tracing sampling chosen consciously · JSON logs, no secrets · graceful shutdown + `preStop` · virtual threads with explicit limits · AOT cache or native where startup matters · secrets from vault/K8s · `MaxRAMPercentage` + right-sized pool · `spring.web.error.include-stacktrace=never` (Boot 4 name) · `ddl-auto=validate` · OSIV off · timeouts on every client · `server.forward-headers-strategy` behind a proxy.

In the interview

Q: What is Actuator and how do you secure it? A: Management endpoints; only `health` exposed by default; include more explicitly, restrict `access`, secure with a `SecurityFilterChain` using `EndpointRequest`, often on a separate port.

Q: Liveness vs readiness? A: Failing liveness restarts the pod; failing readiness removes it from load balancing; a DB check in liveness would restart every pod during a DB outage.

Q: Counter, Gauge, Timer? A: Accumulating events (use `rate()`), current values (queue depth), durations with percentiles; low-cardinality tags.

Q: How does a trace id get from A to B? A: A's instrumented client injects `traceparent`; B's server observation extracts it and creates a child span; both log the trace id via MDC.

Q: Why is my trace broken between services? A: Client created with `new`/`builder()` instead of Boot's builder, a proxy strips headers, W3C vs B3 mismatch, or async code lost the context.

Q: Zero-downtime deploys on Kubernetes? A: Readiness probe, graceful shutdown, `preStop` delay, grace period > drain timeout, rolling update with `maxUnavailable: 0`.

Q: How would you speed up startup? A: Remove slow `@PostConstruct` work, lazy init where safe, extracted jar + AOT cache, or a native image; diagnose with `/actuator/startup`.

Gotchas

- `/actuator/prometheus` 404 → registry dependency missing or not exposed; missing only in tests → `@AutoConfigureMetrics`.
- No trace ids in logs → no tracer on the classpath or custom pattern without `%X{traceId}`.
- High-cardinality tags → Prometheus memory explodes.
- 502s during deploys → no `preStop`, grace timeout too short.
- `show-values=always` → secrets in `/actuator/env`.
- Old `server.error.*` keys → no longer bound in Boot 4 (`spring.web.error.*`).

TL;DR

Actuator (only `health` by default; probes on in Boot 4; nothing external in liveness), Micrometer with low-cardinality tags, Observation → metrics + traces, `traceparent` + 10 % sampling, JSON logs, graceful shutdown + `preStop`, AOT cache or native for startup.

16. Spring Cloud and Microservices

Release trains, Config Server, discovery (Eureka, Consul, Kubernetes), LoadBalancer, OpenFeign vs HTTP interface clients, Gateway, Resilience4j breaker/retry/rate limiter, tracing across services, contract testing.

Must-know

- **Release train** — Boot 4.0/4.1 → **Spring Cloud 2025.1.x "Oakwood"** (GA Nov 2025; Boot 4.1 from **2025.1.2**; 2025.1.3 used in the book), components 5.0.x; import the `spring-cloud-dependencies` BOM. Wrong train → startup compatibility error.
- **Config Server** — `@EnableConfigServer`, Git (or `native`, Vault…) backend; clients `spring.config.import=configserver:` (`optional:` to tolerate absence); precedence `app-profile` > `app` > `application`; `@RefreshScope` (and `@ConfigurationProperties`) beans rebuilt on `POST /actuator/refresh`, all instances via Spring Cloud Bus. On Kubernetes, ConfigMaps/Secrets + `configtree:` often replace it.
- **Discovery** — `DiscoveryClient` over Eureka/Consul/Zookeeper/Kubernetes; on Kubernetes, Services + DNS usually suffice.
- **Spring Cloud LoadBalancer** — resolves service ids (`http://inventory`, `lb://inventory`) per request, **round robin** by default; `@LoadBalanced RestClient.Builder`; add Caffeine for its cache. Client-side (smarter, per request, library) vs server-side (language-agnostic, extra hop).
- **Declarative clients** — new code: `@HttpExchange` interfaces (Boot 4 groups, load-balanced when the group has no base URL); **OpenFeign is feature-complete / maintenance only**.

- **Spring Cloud Gateway** — routes = id + URI + **predicates** (path, host, header) + **filters** (`StripPrefix`, headers, `CircuitBreaker` with fallback, rate limiting, token relay); flavours `spring-cloud-starter-gateway-server-webflux` (Netty) and `-server-webmvc` (servlet, virtual threads). Gateway 5.0: MVC `http(String)`/`http(URI)` gone → `http()` + `before(uri(...))`. Keep business logic out.
- **Circuit breaker** (Resilience4j) — CLOSED records outcomes in a sliding window → failure or slow-call rate over threshold after `minimum-number-of-calls` → OPEN (fail fast) → after `wait-duration-in-open-state` → HALF_OPEN (`permitted-number-of-calls-in-half-open-state` trial calls) → CLOSED or OPEN. Config `resilience4j.circuitbreaker.instances.<name>`; `@CircuitBreaker(name, fallbackMethod)`; artifact `resilience4j-spring-boot4` 2.4.0.
- **Retry vs breaker** — retry repeats one call; the breaker stops calling an unhealthy dependency. In Resilience4j's default aspect order **retry wraps the breaker**.
- **Resilience order** — timeouts first → retries (idempotent, back-off + jitter) → circuit breaker → bulkhead → fallbacks.
- **Consistency across services** — no 2PC/XA; **saga** (local transactions + compensations, orchestrated or choreographed) + outbox + idempotent consumers; eventual consistency.
- **Contract testing** — consumer-driven: **Pact** (polyglot, broker, `can-i-deploy`); Spring Cloud Contract is no longer in the 2025.1.3 BOM and its repository is archived — no new work on it.

Key facts & numbers

Spring Cloud component choices:

Problem	Spring answer	On Kubernetes, often instead
one entry point, edge concerns	**Spring Cloud Gateway**	Ingress / Gateway API, service mesh
find changing instances	Eureka, Consul, Spring Cloud Kubernetes	Services + DNS
spread calls over instances	**Spring Cloud LoadBalancer** (client side)	kube-proxy / mesh
config per environment	**Spring Cloud Config**, Vault, Consul KV	ConfigMaps/Secrets + `configtree:`
failing/slow dependencies	**Resilience4j** (+ Spring Cloud CircuitBreaker), F7 `@Retryable`/`@ConcurrencyLimit`	mesh retries/outlier detection
request across services	Micrometer Tracing + OpenTelemetry	+ mesh telemetry
declarative HTTP	HTTP interfaces (OpenFeign = maintenance)	—
events	Spring Kafka / AMQP, Spring Cloud Stream	—

Train	Codename	Boot	Components
2025.1.x	**Oakwood**	4.0.x, 4.1.x (4.1 from 2025.1.2)	5.0.x
2025.0.x	Northfields	3.5.x	4.3.x (OSS support ended June 2026)
2024.0.x	Moorgate	3.4.x	4.2.x

Need	Framework 7 core	Resilience4j
retry + back-off + jitter	`@Retryable` (`maxRetries` = retries **after** the first call)	`@Retry` (`max-attempts` counts the first call)
bulkhead	`@ConcurrencyLimit` (BLOCK/REJECT)	`@Bulkhead` (semaphore / thread pool)
circuit breaker	**no**	`@CircuitBreaker`
rate / time limiter	no	`@RateLimiter`, `@TimeLimiter`
fallback	no (catch at caller)	`fallbackMethod`
switch	`@EnableResilientMethods`	`resilience4j-spring-boot4` + properties

Code to remember

```
resilience4j.circuitbreaker.instances.inventory:
  sliding-window-size: 10
  minimum-number-of-calls: 5
  failure-rate-threshold: 50
  wait-duration-in-open-state: 10s
  permitted-number-of-calls-in-half-open-state: 2
```

```
@CircuitBreaker(name = "inventory", fallbackMethod = "stockUnknown")
int stock(String sku) { return inventoryApi.stock(sku); }
int stockUnknown(String sku, Throwable t) { return -1; }
```

In the interview

Q: Which Spring Cloud version with Boot 4.1? A: 2025.1.x "Oakwood" (2025.1.2+), via the `spring-cloud-dependencies` BOM; components 5.0.x.

Q: How do services find each other? A: `DiscoveryClient` (Eureka/Consul/Kubernetes) + LoadBalancer resolving service ids per request (round robin); on Kubernetes, Services + DNS.

Q: Explain circuit breaker states. A: CLOSED → OPEN when failure/slow-call rate crosses the threshold → HALF_OPEN after the wait → trial calls decide CLOSED or OPEN; Framework 7 has no circuit breaker.

Q: What does an API gateway do; which flavour? A: Routing by predicates, filters for auth/token relay, rate limiting, rewriting, resilience; WebMVC flavour for servlet teams + virtual threads, WebFlux for reactive/very high concurrency.

Q: How do config changes get picked up? A: `@RefreshScope` / `@ConfigurationProperties` beans rebuilt on `/actuator/refresh` or Bus; plain `@Value` in a singleton is not refreshed.

Q: How do you keep data consistent across services? A: Database per service, sagas with compensations, outbox + idempotent consumers; accept eventual consistency.

Q: Feign or HTTP interfaces? A: HTTP interfaces for new code; OpenFeign is maintenance-only.

Gotchas

- `No spring.config.import property has been defined` → config client without the import.
- `UnknownHostException: inventory` → client not `@LoadBalanced` (or HTTP group has a base URL).
- Breaker never opens → no timeouts, slow calls not recorded, window too large.
- Retry storm → retries in several layers; retry in one layer with jitter, breaker around.
- Everything down when one service is down → synchronous chains without timeouts/fallbacks; use async events.
- Gateway 404 → predicates don't match; DEBUG `org.springframework.cloud.gateway`.

TL;DR

Boot 4.1 ↔ Spring Cloud 2025.1.x Oakwood. Check what Kubernetes already gives you; Config Server + `@RefreshScope`, LoadBalancer round robin, HTTP interfaces over Feign, Gateway routes = predicates + filters, Resilience4j CLOSED → OPEN → HALF_OPEN, Pact for contracts.

17. Spring AI and What's New in Boot 4 / Framework 7

Spring AI 2.0 (`ChatClient`, prompts, advisors, memory, structured output, tools, RAG, MCP) tested offline with a fake model; Boot 3 → 4, Framework 6 → 7 and Security 6 → 7 changes; what Boot 4.1 adds.

Must-know

- **Spring AI** — portable APIs over models and vector stores: `ChatModel` / `ChatClient`, `EmbeddingModel`, `VectorStore`, tool calling, chat memory, MCP; one Boot starter per provider. **Spring AI 2.0.x** (GA June 2026; **2.0.1** current) requires Boot 4 (4.1.x smoothest); 1.1.x is for Boot 3.4/3.5. Import `spring-ai-bom`; set `spring.ai.<provider>.api-key` and the chat model explicitly.
- **`ChatModel` vs `ChatClient`** — provider-level interface vs fluent API with templates, advisors, tools, structured output: `chatClient.prompt().user("...").call().content()` / `.entity(Invoice.class)` / `.stream()`. One `ChatClient` bean per use case, built from the injected `ChatClient.Builder`.
- **Prompts** — messages (system, user, assistant, tool); `PromptTemplate` fills `{placeholders}` and fails on missing values; user text never in the system message (prompt injection).

- **Advisors** — wrap every call, ordered (lower first): logging, memory, RAG, tool calling (auto-registered in 2.0), your own (redaction). Memory before RAG/logging; redaction before anything that stores or sends.
- **Memory** — the model is stateless; an advisor stores messages per `ChatMemory.CONVERSATION_ID` and re-sends a window (**20** by default); id derived from the authenticated user; JDBC repository in a cluster.
- **Structured output** — `.entity(MyRecord.class)`: JSON-schema instructions + Jackson parsing (native mode where supported); validate the values like any untrusted input.
- **Tools** — `@Tool` methods; the **application** executes them — the model only requests; `ToolCallingAdvisor` loops (limits 40/150); authorize inside tools; never expose destructive operations unchecked.
- **RAG** — ingest: reader → splitter → `EmbeddingModel` → `VectorStore` (PGVector, Redis…); query: `QuestionAnswerAdvisor` similarity search with a **threshold** and metadata filters (tenant/role) adds chunks to the prompt.
- **MCP** — standard for sharing tools/resources across processes; client and server starters, `@McpTool`; Streamable HTTP is the default web transport in 2.0 (SSE deprecated); client tools arrive as a `ToolCallbackProvider`.
- **Testing** — a fake/stub `ChatModel` bean (fast, deterministic, no key; 14 offline tests in the book); separate evaluations against the real model checking properties (`RelevancyEvaluator`, `FactCheckingEvaluator`).
- **Framework 7** — JSpecify, API versioning, HTTP service groups, `BeanRegistrar`, `@Retryable`/`@ConcurrencyLimit`/`RetryTemplate`, `@Proxyable`, `RestTestClient` + context pausing, Jackson 3 (Jackson 2 deprecated), JUnit 4 support deprecated, `javax.annotation`/`javax.inject` and `spring-jcl` removed, `PathPattern` replaces `AntPathMatcher` for MVC, `HttpHeaders` no longer a `MultiValueMap`, `JmsClient`, `RestTemplate` deprecation planned for 7.1.
- **Security 7** — lambda DSL only, `authorizeHttpRequests`, `PathPatternRequestMatcher`, MFA (`FACTOR_PASSWORD`/`FACTOR_OTT`/`FACTOR_WEBAUTHN`, `@EnableMultiFactorAuthentication`), `csrf.spa()`, Password4j encoders, Authorization Server inside, password grant removed, PKCE default; 7.1 adds `RestClientOpaqueTokenIntrospector`.
- **Boot 4.1 adds** — removes everything deprecated in 4.0; gRPC support; Spock back; `spring.datasource.connection-fetch=lazy`; `Optional` record parameters bind as `Optional.empty()`; `jwt.authorities-claim-expressions`; context propagation into `@Async`; `spring.http.clients.cookie-handling`, `InetAddressFilter` (SSRF); `@RedisListener`; Hibernate 7.4, Spring Data 2026.0, Micrometer 1.17; `layertools` jar mode removed (use `tools`); LiveReload and Derby deprecated.

Key facts & numbers

Boot 3 → 4 migration checklist:

Step	Do
Before	upgrade to **Boot 3.5.x**, zero deprecation warnings (4.0 deletes them); Java 17+ and `jakarta.*`; Kotlin 2.2+, GraalVM 25+, Gradle 8.14+
Automate	OpenRewrite `org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0`; `spring-boot-properties-migrator` temporarily; `spring-boot-starter-classic` / `-test-classic` if stuck
Starters	`-web` → `-webmvc`; `-aop` → `-aspectj`; `-oauth2-*` → `-security-oauth2-*`; `-web-services` → `-webservices`; add `-restclient`, `-webclient`, a `-test` starter per technology, `-flyway`/`-liquibase` (else migrations silently stop)
Servers / libs	Undertow → Tomcat/Jetty; Spring Retry no longer managed → declare version or F7 `@Retryable`; Spring Cloud **2025.1.x** (2025.1.2+ for 4.1), Spring AI **2.0**, springdoc **3.x**, Testcontainers **2.x** artifact names
Jackson 3	`com.fasterxml.jackson.databind` → `tools.jackson.databind` (annotations unchanged); `ObjectMapper` → `JsonMapper`; `JsonMapperBuilderCustomizer`; `@JacksonComponent`, `@JacksonMixin`; `spring.jackson.use-jackson2-defaults=true` as a bridge
Code	auto-config class packages moved (check `exclude`); `@EntityScan` → `org.springframework.boot.persistence.autoconfigure`; `hibernate-jpamodelgen` → `hibernate-processor`; JSpecify `@Nullable`; Security `.and()` → lambdas; `RestTemplate` → `RestClient`
Tests	`@MockBean`/`@SpyBean` → `@MockitoBean`/`@MockitoSpyBean`; `@AutoConfigureMockMvc` / `RestTestClient` / `WebTestClient` / `TestRestTemplate` explicit; slice imports moved; JUnit 4 → JUnit 6
Behaviour	health probes on by default; LiveReload off; `server.forward-headers-strategy` no longer for WARs; Jackson 3 defaults (ISO dates, unknown properties ignored); embedded launch scripts removed

Boot 3 property	Boot 4 property
`spring.jackson.parser.*` / `generator.*`	`spring.jackson.json.read.*` / `json.write.*`
`spring.data.mongodb.uri`	`spring.mongodb.uri`
`spring.http.client.*`	`spring.http.clients.*`
`server.error.*`	`spring.web.error.*`
`spring.dao.exceptiontranslation.enabled`	`spring.persistence.exceptiontranslation.enabled`
`management.tracing.enabled`	`management.tracing.export.enabled`
`management.otlp.tracing.endpoint`	`management.opentelemetry.tracing.export.otlp.endpoint`
`management.health.mongo.enabled`	`management.health.mongodb.enabled`
`spring.session.redis.*`	`spring.session.data.redis.*`
`spring.kafka.retry.topic.backoff.random`	`...backoff.jitter`
`spring.rabbitmq.listener.simple.retry.max-attempts`	`...retry.max-retries` (excludes the first call)

Baseline	Boot 3.x	Boot 4.x
Framework / Jakarta EE	6.x / 10 (Servlet 6.0)	7.0 / **11** (Servlet 6.1, JPA 3.2, BV 3.1)
Servers	Tomcat 10.1, Jetty 12, Undertow	Tomcat 11, Jetty 12.1, **no Undertow**
Hibernate / JUnit	6.x / 5	7.x (7.4) / **6**

Code to remember

```
@Bean
ChatClient support(ChatClient.Builder builder, ChatMemory memory) {
    return builder.defaultSystem("You answer questions about orders.")
                  .defaultAdvisors(MessageChatMemoryAdvisor.builder(memory).build())
                  .build();
}
Invoice inv = support.prompt().user(text)
        .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, userId))
        .call().entity(Invoice.class);                  // structured output → record
```

In the interview

Q: What is Spring AI; which version with Boot 4? A: Portable model/vector-store APIs with Boot starters, advisors for memory, RAG, tools; 2.0.x (2.0.1) for Boot 4, 1.1.x for Boot 3.4/3.5.

Q: How does chat memory work? A: The model is stateless; an advisor stores messages per conversation id and re-sends a window (20 by default).

Q: Who executes a tool? A: The application — the model only requests the call; `ToolCallingAdvisor` runs it and loops; authorize inside the tool.

Q: How does RAG work? A: Chunk → embed → store; at query time a RAG advisor runs a similarity search (threshold, metadata filters) and adds the best chunks to the prompt.

Q: How do you test LLM code? A: Fake `ChatModel` with canned answers for logic; real-model evaluations of relevance/grounding/format separately.

Q: What changed Boot 3 → 4? A: Framework 7/Jakarta EE 11, Undertow removed, modular starters + test starters, Jackson 3, JUnit 6, JSpecify, API versioning and HTTP service client auto-config, `RestTestClient`, OpenTelemetry starter, probes on by default, `@MockBean` removed, explicit `@AutoConfigure…` for test clients.

Q: How would you migrate? A: 3.5.x with no deprecations → parent 4.x → rename starters (or `-classic` temporarily) → Jackson 3 → `@MockitoBean` + test starters → Spring Cloud 2025.1.x, Spring AI 2.0 → property renames → run tests, compare condition report and `/actuator`; OpenRewrite does the mechanical work.

Gotchas

- One fixed conversation id for everyone → users see each other's history.
- In-memory chat memory in a cluster → different per instance, lost on restart.
- Trusting the parsed object → right shape, wrong values; validate.

- No similarity threshold / no permission filters in RAG → irrelevant or leaked chunks; don't mix embedding models in one store.
- Calling the model inside a DB transaction → connection held for seconds.
- Logging full prompts → personal data (observations keep prompts out by default).
- Migration: forgetting `spring-boot-starter-flyway` → migrations silently stop; no `-webclient` starter → "No qualifying bean of type WebClient$Builder".

TL;DR

Spring AI 2.0.x: `ChatClient` + advisors (memory, RAG, tools), `.entity(...)` for typed output, `@Tool`/MCP, test with a fake model. Boot 4 migration: via 3.5, renamed + per-tech test starters, Jackson 3, `@MockitoBean`, JSpecify, Security 7 lambda DSL, Spring Cloud 2025.1.x.

18. Spring Interview Questions

The book's 168 questions (already folded into every "In the interview" above and the Top 25), five tricky answers proved by tests, the project walkthrough and ten debugging scenarios.

Must-know

- **Three-layer answer** — what it is → how it works underneath → the consequence that bites (layer 3 shows you've debugged it).
- **Five proved tricky answers** — prototype in a singleton is created once; `@Qualifier` > `@Primary` > parameter name; constructor cycles fail (Boot also rejects field cycles); a checked exception **commits**; self-invocation skips `@Transactional`.
- **Debugging shape** — symptom → cause (the mechanism, not a guess) → smallest fix → proof (log line, test, metric). Read the **root cause** at the bottom of a Spring stack trace.
- **Project walkthrough (~5 min)** — purpose → scale → architecture picture → stack with one reason each → one request end to end → data and consistency → security → tests → running it → one hard problem (STAR) + one improvement. Say what **you** did; don't mention what you can't explain two levels deeper; don't invent numbers.

Key facts & numbers

The ten debugging scenarios:

Symptom	Likely cause	Fix	Prove it
"No qualifying bean" / "required a single bean, but 2 were found"	outside scan, missing starter/profile, backed-off auto-config; two candidates	move main class, add starter, `@Qualifier` / `@Primary`	`--debug` report, `/actuator/beans`, `ApplicationContextRunner`
"beans form a cycle"	A ↔ B constructors (or fields, rejected since 2.6)	third bean or event; `@Lazy` patch	context test starts cleanly
`@Transactional` doesn't roll back	checked exception, caught exception, self-invocation/`private`, wrong manager, other thread	`rollbackFor` / `ALL_EXCEPTIONS`, rethrow, extract bean	`isActualTransactionActive()`, TRACE `transaction.interceptor`, row-count test
`LazyInitializationException`	lazy association after the transaction (often after OSIV off)	fetch in query, DTOs inside the transaction	`@DataJpaTest` reproduces, SQL log
list endpoint slow, hundreds of SELECTs	N+1	`JOIN FETCH` / `@EntityGraph` /DTO; `default_batch_fetch_size=16`	statement-count assertion
unexpected 401 / 403	expired/invalid token vs missing `ROLE_`, CSRF, rule order, wrong chain, `/error` not permitted	fix mapping, order rules, permit `/error`	`org.springframework.security=TRACE`, security tests
"blocked by CORS policy" (curl works)	preflight hits security (401), origin mismatch, credentials + `*`, header added twice	one `CorsConfigurationSource` + `http.cors()`, one layer	replay `OPTIONS` with curl, preflight test
`@Async` / `@Cacheable` / `@Transactional` does nothing	call bypassed the proxy or `@Enable…` missing	separate bean, public non-final, enable	`AopUtils.isAopProxy`, thread name, call counts
context fails after adding a starter	auto-config needs settings (DataSource URL), bean name clash, version fights with BOM	provide config / exclude / drop explicit versions	failure-analyzer Description/Action, `mvn dependency:tree`
slow startup (48 s)	heavy `@PostConstruct`, hanging remote calls, big migrations, many test contexts	move warm-up to `ApplicationReadyEvent`, timeouts, standardise tests	`BufferingApplicationStartup` + `/actuator/startup`, cache DEBUG log

In the interview

Q: Postman works, the browser doesn't — first suspicion? A: CORS: usually the preflight hitting security (401) or an origin mismatch.

Q: Which log tells you why a request got 403? A: `org.springframework.security` at TRACE — every filter and the authorization decision.

Q: Quickest proof that N+1 is fixed? A: A test asserting the SQL statement count.

Q: A new starter made startup fail — where first? A: The failure analyzer's Description/Action, then the `--debug` condition report.

Q: "Why not WebFlux?" / "What if Kafka is down?" / "How do you avoid double charging?" A: Virtual threads give the concurrency with simple code; the outbox keeps events and the relay retries; idempotency keys with a unique constraint + idempotent consumers.

Gotchas

- Listing technologies without reasons → give one reason per choice.
- Guessing a cause → name the mechanism and show the evidence.
- `spring.main.allow-circular-references=true` as a "fix" → hides the design problem.

TL;DR

Answer in three layers; debug as symptom → mechanism → smallest fix → proof; walk through your project from purpose to one hard problem.

19. Cheat Sheet and Glossary

Versions, annotations, starters, properties, and "which tool for which job" — its tables are folded into the chapters above (scopes and lifecycle → Ch 2, conditions → Ch 3, propagation → Ch 7, slices → Ch 12, Reactor → Ch 13, Actuator → Ch 15, migration → Ch 17). The 343-term glossary is skipped here.

Must-know

- **One number pins everything** — `spring-boot-starter-parent` **`4.1.1`**; its BOM chooses the rest. Never add `<version>` to managed dependencies.
- **Not in the Boot BOM** — Spring Cloud 2025.1.x, Spring AI 2.0.x, springdoc 3.x, `resilience4j-spring-boot4` 2.4.0: import their BOMs or set versions.

Key facts & numbers

Component	Boot 4.1.1 BOM
Spring Framework	7.0.9
Spring Security	7.1.1 (Authorization Server included)
Spring Data	2026.0
Hibernate ORM / Validator	7.4.5 / 9.1.3
Jackson	3.1.5
Tomcat / Jetty	11.0.24 / 12.1.12
HikariCP / PostgreSQL JDBC	7.0.2 / 42.7.13
Reactor	2025.0 (Core 3.8)
Micrometer / Tracing / OpenTelemetry	1.17.1 / 1.7.1 / 1.62.0
Spring Kafka / kafka-clients / Spring AMQP	4.1.1 / 4.2.1 / 4.1.1
Flyway / Liquibase	12.4.0 / 5.0.3
JUnit / Mockito / AssertJ / Testcontainers	6.0.3 / 5.23.0 / 3.27.7 / 2.0.5
Caffeine / Lettuce	3.2.4 / 7.5.2

Which tool for which job:

Question	Answer
`@Value` or `@ConfigurationProperties`?	one value/SpEL → `@Value`; a group → a validated record
`@Component` or `@Bean`?	your class → stereotype; third-party or creation logic → `@Bean`
unit, slice or `@SpringBootTest`?	logic → unit; one layer → slice; wiring → a few full tests
`Page`, `Slice` or `Window`?	page numbers → `Page`; "load more" → `Slice`; big tables → keyset `Window`
`@EventListener`, `@TransactionalEventListener` or outbox?	in-process now → listener; after commit → transactional listener; must not be lost → outbox
JWT or opaque?	local fast validation → JWT; instant revocation → opaque
session cookie or tokens?	browser/BFF → session + CSRF; service-to-service, mobile → bearer
blocking or non-blocking Kafka retries?	order matters → `DefaultErrorHandler`; independent → `@RetryableTopic`
Flyway or Liquibase?	SQL-first, one DB → Flyway; rollbacks, preconditions, many DBs → Liquibase
Eureka or Kubernetes Services?	on Kubernetes, Services + DNS usually suffice
JVM + AOT cache or native image?	long-running, throughput → JVM (AOT cache halves startup); CLIs, scale-to-zero → native

Gotchas

- Defaults to remember: `spring.main.allow-circular-references=false`, `allow-bean-definition-overriding=false`, `lazy-initialization=false`, `spring.jpa.open-in-view=true` (turn off), `spring.threads.virtual.enabled=false`, test context cache max 32.

TL;DR

Boot 4.1.1 pins Framework 7.0.9, Security 7.1.1, Hibernate 7.4, Jackson 3.1; Spring Cloud 2025.1.x and Spring AI 2.0.x come from their own BOMs.

Last-minute checklist

- Boot 4.1.1 → Framework 7.0.9, Security 7.1.1, Hibernate 7.4, Jackson 3 (`tools.jackson`), JUnit 6; Java 17 minimum (25 recommended); Spring Cloud 2025.1.x Oakwood; Spring AI 2.0.1.
- `@SpringBootApplication` = configuration + auto-configuration + component scan from the main package down.
- Bean choice: type → `@Qualifier` → `@Primary` → `@Fallback` → parameter name; singleton = shared = stateless.
- Prototype in singleton = created once → `ObjectProvider`; constructor cycles always fail.
- Lifecycle: construct → inject → Aware → BPP-before → `@PostConstruct` → `afterPropertiesSet` → init → BPP-after (**proxies**) → `@PreDestroy` → `destroy`.
- Auto-configuration = `.imports` + conditions + `@ConditionalOnMissingBean`; explain with `--debug`.
- Property precedence: command line > env vars > profile files > `application.*`; outside jar > inside.
- Proxies: self-invocation, `private`/`final`, `new`, missing `@Enable…` skip `@Transactional`/`@Cacheable`/`@Async`/`@Retryable`/`@PreAuthorize`.
- MVC: filters → `DispatcherServlet` → mapping → interceptors → args + validation → controller → converter → exception resolvers; errors as RFC 9457 `ProblemDetail`.
- Clients: `RestClient` from the injected builder or `@HttpExchange` + `@ImportHttpServices`; always timeouts; `RestTemplate` deprecation 7.1, removal 8.0.
- Transactions: REQUIRED + DB-default isolation; checked exceptions **commit**; caught exceptions never roll back; `REQUIRES_NEW` = second connection; `UnexpectedRollbackException` = caught inner failure.
- JPA: everything LAZY; N+1 → `JOIN FETCH`/`@EntityGraph`/batch size/DTOs; OSIV off; `@Version`; Flyway with `spring-boot-starter-flyway`.
- Caching: default key has no method name; immutable values; evict after commit; `sync = true` is per JVM.
- Security: first matching chain, first matching rule; `ROLE_` prefix; **401 = who are you, 403 = no**; CSRF on with cookies; lambda DSL only in 7.
- JWT resource server: JWKS by `kid`, `exp`/`nbf` (60 s), `iss`, `aud`; `scope` → `SCOPE_x`; BFF for browsers.
- Tests: unit → slices → few `@SpringBootTest`s; `@MockitoBean`; `@AutoConfigureMockMvc`/`RestTestClient` explicit in Boot 4; cache max 32, paused contexts.

- Reactive: lazy until subscribe; `flatMap` concurrent vs `concatMap` ordered; never block event loops; MVC + virtual threads 7,942 ≈ WebFlux 8,058 req/s ≫ 1,909 platform.
- Async/scheduling: `@EnableAsync`/`@EnableScheduling`; 8-thread executor with unbounded queue; one scheduler thread; jobs run on every instance.
- Kafka: order per partition (key = aggregate id); at-least-once → idempotent consumers; `DefaultErrorHandler` + `<topic>-dlt`; `@RetryableTopic` loses order; outbox for DB + event.
- Actuator: only `health` exposed; probes on in Boot 4; no DB in liveness; tracing sampling 0.1; graceful shutdown 30 s + `preStop`.
- Cloud: Config + `@RefreshScope`, LoadBalancer round robin, HTTP interfaces > Feign, breaker CLOSED → OPEN → HALF_OPEN, no breaker in Framework 7, Pact for contracts.
- Migration: 3.5 first → renamed/test starters → Jackson 3 → `@MockitoBean` → property renames (`spring.http.clients.*`, `spring.web.error.*`, `spring.mongodb.*`).
- Debug: symptom → mechanism → smallest fix → proof.